

# Threshold Niederreiter: Chosen-Ciphertext Security and Improved Distributed Decoding

Pascal Giorgi<sup>1</sup>, Fabien Laguillaumie<sup>1</sup>, Lucas Ottow<sup>1</sup>, and Damien Vergnaud<sup>2</sup>

<sup>1</sup>Univ. Montpellier, LIRMM, Montpellier, France

<sup>2</sup>Sorbonne Université, CNRS, LIP6, F-75005 Paris, France

**Abstract.** Threshold public-key encryption securely distributes private key shares among multiple participants, requiring a minimum number of them to decrypt messages. We introduce a quantum-resistant threshold public-key encryption scheme based on the code-based Niederreiter cryptosystem that achieves security against chosen ciphertext attacks. A previous attempt was made recently by Takahashi, Hashimoto, and Ogata (published at DCC in 2023) but we show that it contains a critical security flaw that allow adversaries to exploit malformed ciphertexts to gain information about the secret key.

In this work, we formalize a generic conversion enhancing security of (classical) public-key encryption from one-wayness against passive attacks to indistinguishability against chosen-ciphertext attacks. The conversion uses a non-interactive zero-knowledge argument with strong security properties to ensure ciphertext well-formedness. We then provide an instantiation for Niederreiter encryption based on recent techniques introduced in the "MPC-in-the-head" paradigm. The publicly verifiable validity of ciphertexts makes this scheme suitable for threshold public-key encryption and prevents an attack similar to the one on Takahashi-Hashimoto-Ogata scheme. To improve the multi-party computation protocol for decryption (involving secure computations on polynomials), we introduce a field-switching technique that allows to significantly reduce the shared secret key size and computational overhead.

## 1 Introduction

Threshold public key encryption [DF90, Des93] is a type of cryptographic primitive that is similar to public key encryption (PKE), except that the private key is securely divided into shares that are distributed among  $N$  participants. In such a threshold scheme, a least a certain number  $T$  of the participants is required to successfully decrypt a ciphertext. Any set of at most  $T - 1$  parties should not be able to learn any information about the plaintext.

Many threshold encryption schemes exist in the literature. The discrete logarithm case has been addressed for instance in [FP01, LP01, SG02, GJKR07], RSA in [DDFY94, GRJK00], Paillier in [FPS01, DJ01, HMR<sup>+</sup>19] or Castagnos-Laguillaumie in [CLT22, BCD<sup>+</sup>24]. However, all those schemes rely on assumptions that are not resistant to quantum computers. The need is important for

quantum-resistant threshold cryptography primitives, so much so that the NIST made a call<sup>1</sup> in 2024 for multiple categories of threshold schemes, one of which being for quantum-resistant threshold PKEs. In the realm of lattice-based cryptography, a number of works exist on threshold PKEs, the work of Bendlin and Damgård [BD10] being the first to deal with this topic. More recently, threshold lattice-based constructions with varying properties have been proposed: For example, schemes that achieve adaptive security [DLN<sup>+</sup>21], or that are fully homomorphic [BS23].

**Existing solutions relying on error correcting codes.** Threshold PKEs based on other post-quantum assumptions than lattices are currently not well studied. Takahashi, Hashimoto and Ogata [THO23] were the first authors attempting to design threshold PKEs based on assumptions from error-correcting codes. In their article, three new schemes are presented. All of them rely on the same assumption as the Classic McEliece KEM-DEM submission at the NIST PQC competition [ABC<sup>+</sup>22].

To design each of the schemes, the authors start with the (non-threshold) OW-CPA PKE variant of the Niederreiter cryptosystem [Nie86] that is used in the Classic McEliece KEM-DEM. Then, for each scheme, they apply different conversions to obtain a threshold IND-CCA scheme. The first proposition uses a non-threshold IND-CCA conversion of the Niederreiter cryptosystem [BHH<sup>+</sup>19], and then apply a generic conversion by Dodis and Katz [DK05] to obtain a threshold scheme. The size of the ciphertexts grows linearly in the number  $N$  of participants. Moreover, each ciphertext for this scheme has a signature component, which increases its size. For their second variant, the authors propose their own generic conversion from an OW-CPA scheme into a threshold OW-CPA scheme. Then, they apply the conversion from Cong, Cozzo, Maram, and Smart [CCMS21] to obtain an IND-CCA threshold scheme. However, the resulting ciphertexts and secret keys are of size  $\mathcal{O}(\binom{N}{T-1})$  (recall that  $T$  is the minimal number of participants for decryption). Both of these constructions are impractical for applications where the number of participants  $N$  is large,.

To build their third scheme, the authors propose a secure multi-party computation (MPC) protocol that carries out the same operation (decoding Goppa codes) as the decryption algorithm for the Niederreiter cryptosystem. This leads to a threshold OW-CPA scheme, that is then converted into an IND-CCA scheme using the same conversion of [CCMS21]. This scheme is the only one of the three for which the size of the ciphertexts does not depend on  $N$  or  $T$ , and is the only one that uses a conversion specifically designed for the Niederreiter cryptosystem.

**A security flaw.** However, the conversion from [CCMS21] is used incorrectly on the third scheme from [THO23], and therefore the resulting PKE does *not* achieve IND-CCA2 security if the adversary corrupts participants of the decryption protocol. Indeed, if the multi-party decoding is fed with an incorrectly

<sup>1</sup> <https://csrc.nist.gov/projects/threshold-cryptography>

formed ciphertext, *i.e.* a vector of incorrect Hamming weight, the MPC protocol stops prematurely, and reveals information correlated to the secret key. This is due to the fact that, during the multi-party decoding (which is the core part of the distributed decryption), the authors of [THO23] propose to build and invert a shared subresultant matrix between a secret polynomial  $g$  that is part of the secret key, and a polynomial depending on the ciphertext. However, their method for inverting matrices do not check in any way if the shared matrix is indeed invertible. If this shared matrix is not, its rank can be deduced by all the parties during the computation and the MPC protocol fails. Moreover, this subresultant matrix is precisely invertible if and only if the input of the decoding algorithm has an error of the correct Hamming weight  $t$ . In the conversion of [CCMS21], no preliminary checks or conditions are done *before* carrying out the multi-party decryption. This means that the adversary can submit a ciphertext containing a (potentially malformed) Niederreiter ciphertext  $He$  ( $H$  is a matrix corresponding to a public key of the Niederreiter cryptosystem), and know whether  $e$  is a vector of Hamming weight  $t$ . This leads to a threshold "OW-CCA2" attack on the third scheme of [THO23]: If an adversary wants to know the value of an error vector  $e^*$  given an encryption  $He^*$  corresponding to the challenge ciphertext, it can simply compute  $He^* + H\delta_i$ , where  $\delta_i$  is the zero vector, except that the first and  $i$ th coordinate are 1. Then, the adversary can ask for a threshold decryption and participates in the multi-party decoding, since no preliminary check is made on the ciphertext before carrying out the decoding protocol. If the MPC protocol succeeds, then it means that  $\text{wt}(e^* + \delta_i) = t$ , and thus  $e_1^* \neq e_i^*$  ( $e$  has coefficients in  $\mathbb{F}_2$ ). If the protocol fails, then  $\text{wt}(e^* + \delta_i) \neq t$ , and  $e_1^* = e_i^*$ . This attack recovers  $e^*$  entirely up to the value of the first coordinate (which is either 0 or 1) in only  $n - 1$  decryption queries. This problem does not seem fixable easily by modifying the multi-party decoding protocol and then applying the conversion from [CCMS21] with the same performance. Indeed, checking if a matrix is invertible in a secure distributed way is significantly more costly than inverting a matrix that is known to be invertible.

Furthermore, some details in the MPC protocol in [THO23] for decoding are missing. For example, once the parties obtain the evaluation of the error-locating polynomial in shared form, it is unclear how the zero-testing of the evaluation is carried out. Their protocol is also inefficient in terms of private key size. While the non-threshold variant of the Niederreiter cryptosystem has a key size of  $n + t$  elements in  $\mathbb{F}_{2^m}$ , their threshold variant has a shared key size of  $\mathcal{O}(nt + t^2)$  elements  $\mathbb{F}_{2^m}$  (meaning each participants hold  $\mathcal{O}(nt + t^2)$  elements as their partial secret key). These extra elements are precomputations depending only on the secret key aimed at making the MPC protocol for decryption faster but make storage of the partial secret keys impractical. Finally, their MPC protocol for decryption is not in a constant number of rounds of communication. Indeed, in order to compute the Hamming weight in shared form, the authors propose to compute a full adder circuit in MPC, which is not of constant depth.

**Our contribution.** We propose an IND-CCA2 threshold scheme based on the Niederreiter cryptosystem, for which the size of the keys as well as the ciphertext expansion does not depend on the total number  $N$  of participants or the minimal number  $T$  of people necessary to decrypt. Our scheme is the most efficient IND-CCA code-based threshold scheme for a high number  $N$  of participants. Our secret keys sizes are very close to the one in the original Niederreiter scheme (only tens of bytes of difference), and our ciphertext sizes are tens of kilobytes, regardless of  $N$ . This size of ciphertext is competitive with all of the schemes in [THO23] that achieves IND-CCA security when considering the "Large" regime ( $N \geq 64$ ) and above from the NIST call for multi-party threshold schemes.

To build our scheme, we provide a generic conversion from a OW-CPA to an IND-CCA (non-threshold) scheme inspired from the Naor-Yung conversion [NY90]. Along each ciphertext, a non-interactive zero-knowledge (NIZK) proof is required, which ensures that the ciphertext is correctly built. Since *all* participants can check this proof *before* participating in the multi-party decryption, this ensures that the multi-party decoding protocol will never run on incorrect inputs. This solves the security issue of the protocol from [THO23]. This proof system needs to abide by a variant of straight-line extractability for the generic conversion to work. We call this property *(one-time) simulation straight-line extractability*. To instantiate the conversion for the Niederreiter cryptosystem, we build a NIZK proof system that abides by this property that proves the fact that a Niederreiter ciphertext is correctly built. Finally, we provide a refined MPC protocol for decryption compared to the third scheme of [THO23]. We improve significantly the size of the shared secret key (going from a few hundred kilobytes to just less than six kilobytes) and we clear up some undisclosed operations in [THO23].

**Overview of our techniques.** To achieve IND-CCA2 security, our conversion requires a ciphertext to be sent along a NIZK proof that the message satisfies some relation. In the Naor-Yung conversion [NY90] and its variants (Sahai [Sah99] and [Lin03b]), two ciphertexts of the same message under two different keys are sent along the proof that the two ciphertexts correspond to the same message. Our techniques only need to encrypt the message once and build a NIZK proof that the ciphertext is constructed correctly. Our proof system needs to meet a stronger security property than in [Sah99] and [NY90], named (one-time) simulation straight-line extractability. Straight-line (or online) extraction refers to the fact that the witness of language membership can be extracted from a prover only by looking at its requests to a random oracle. This property can be found in [Fis05], and the property of straight-line extraction was the baseline for our variant. Straight-line extractability is a stronger notion than special soundness, as having a rewindable black-box access to the adversary allows in particular to look at its requests to the oracle. To achieve IND-CCA2 security (instead of IND-CCA1 as in [NY90]), Lindell [Lin03a] needed the proof to be simulation sound, so that the adversary could not forge a proof without a witness, even after having seen a simulated proof. We need a similar property to

achieve IND-CCA2 security. Our notion of simulation straight-line extractability fuses the notions of straight-line extractability and simulation soundness to achieve IND-CCA2 security with only one ciphertext.

To apply this generic conversion to the Niederreiter cryptosystem, we need a NIZK proof system that can prove that a Niederreiter ciphertext is correctly built and that satisfies our security notion. We build a variant of the proof system by Feneuil, Joux, and Rivain [FJR22]: The goal of their proof is, given a matrix  $H$  and a vector  $y$ , to prove the knowledge of a solution to the syndrome decoding problem, *i.e.* a vector  $r$  such that  $Hr = y$  and Hamming weight  $\text{wt}(r) \leq t$ . Their work uses the “MPC-in-the-Head” (MPCitH) paradigm, which consists for the prover to emulate an MPC protocol locally, committing to this protocol, and then revealing the views of the simulated players except one. The hope is that if the revealed views are consistent, then the prover did cheat only with low probability. In their MPCitH protocol, they verify one polynomial equation that corresponds to verifying that the vector  $r$  is of Hamming weight  $\leq t$ . In our variant, we add another polynomial equation that verifies the vector  $r$  is of Hamming weight  $\geq t$ . If we combine the two inequalities on the Hamming weight of  $r$ , this corresponds exactly to a well-built ciphertext if  $H$  is the public key corresponding to a Niederreiter and  $y$  is a (potential) ciphertext under  $H$ . We prove that our variant of the proof system is indeed simulation straight-line extractable, and can be used to apply our generic conversion to the Niederreiter cryptosystem. Other proof systems for the knowledge of a vector of given Hamming weight exist, like [Ste94,FJR23,MGS11], but our choice of the one from [FJR22] was driven by the fact that these other proofs were not likely to be online extractable, and because [FJR22] has other appealing properties, like the fact that techniques from [AGH<sup>+</sup>23] can be applied to make it more efficient computationally wise. Moreover, some proof systems that are used to build short signatures (as in [BBGK24]) leverage the fact that the parameter set for Syndrome Decoding (on which the soundness relies) can be chosen for the proof system. This is not applicable here, as our proof system is constrained by the Classic McEliece parameters set.

Finally, we are able to improve the MPC protocol for decoding of [THO23] in several ways. First, we manage to reduce the size of the shared secret key from  $\mathcal{O}(nt + t^2)$  to  $n + 3t$  elements of  $\mathbb{F}_{2^m}$  while keeping the same communication complexity (which is  $\mathcal{O}(t^3 + nt)$ ). We achieve this by using a technique that we call *field switching*, which allows us to avoid the use of large precomputed values within the first step of the distributed decoding process. More precisely, decoding the syndrome implies computing many inverses and a square root in a finite field defined with a shared irreducible polynomial  $[g]$  of degree  $t$  that is part of the secret key. Instead of computing these operations modulo  $[g]$ , which is either costly or requires large precomputation to be efficient, we revert to perform this operation modulo a public irreducible polynomial  $h$  (of degree  $t$ ) for which the operations can be done efficiently. To evaluate the embedding between these two fields, we rely on a well-chosen modular composition [Sho94,BDFD<sup>+</sup>19] that can also be computed efficiently and only require  $2t$  precomputed values. For these

first steps, our MPC protocol requires  $\mathcal{O}(t^2)$  secure multiplications compared to  $\mathcal{O}(t^3)$  in [THO23]. This means that if an efficient MPC protocol for computing rational reconstruction is found in the future, the communication complexity of decoding Goppa would drop from  $\mathcal{O}(t^3 + nt)$  to  $\mathcal{O}(t^2 + nt)$  secure multiplications. We also correct the unspecified steps of the protocol. Notably, equality testing in a characteristic of size 2 is done by revealing in the clear the result of the test. This means that at the end of the computation, all participants know the result of the decoding, instead of a shared form.

## 2 Preliminaries

We denote by  $\lambda$  a security parameter. For a probabilistic algorithm  $\mathcal{A}$  and an oracle (or a list of oracles)  $O$ , we denote by  $\mathcal{A}^O$  an execution of  $\mathcal{A}$  with access to the oracle (or all the oracles in the list)  $O$ . A similar notation is for example used in [Fis05]. In [Fis05], algorithms that are allowed to modify the values of random oracles (such as simulators for the zero-knowledge property) return a modified version of the oracle, and the new version is used as input for all subsequent algorithms of the security game. Here, for convenience, we denote by  $\mathcal{A}^{O_{write}}$  an execution of  $\mathcal{A}$  for which  $\mathcal{A}$  has the ability to modify a value of  $O$  at a preimage that has not been queried before.

### 2.1 Security Models for Multiparty Computation

Let  $N \in \mathbb{N}$ . An interactive  $N$ -party protocol  $\Pi$  is defined by  $N$  interactive algorithms  $(\Pi_1, \dots, \Pi_N)$ . The security of such protocols is evaluated against different adversarial models. Two commonly discussed models are the *semi-honest* (or honest-but-curious) model and the *malicious* model.

*Semi-Honest Model.* In the semi-honest model, the adversary is a subset of the participating parties that (1) may analyze their input and output, along with all communication during the protocol, (2) may use this knowledge to infer information about the inputs of honest parties and (3) has to behave according to the protocol, without introducing errors or deviations.

*Malicious Model.* In the malicious model, the adversary may again be any subset of the participants and can, in addition, exhibit arbitrary behavior, such as providing false inputs or messages to disrupt the computation, altering messages sent to other parties to mislead them, and collaborating with other malicious parties to gain more information or manipulate the computation.

*Ideal vs. Real World Simulation Paradigm.* In secure multiparty computation, an  $N$ -party protocol  $\Pi$  computing an  $N$ -ary function  $f$  is deemed  $T$ -private (for some  $T < N$ ), if a simulator can recreate the view of up to  $T$  compromised parties in a real execution, using only the inputs and outputs of those corrupted parties. This is formally defined through two different worlds:

- **Real World:** A real-world execution of the protocol  $\Pi$  in the presence of an adversary  $\mathcal{A}$ , on inputs  $(x_1, \dots, x_N)$  is denoted as:  
 $\text{EXEC}_{\text{real}, \mathcal{A}}(x_1, x_2, \dots, x_N) \rightarrow (y, \text{View}_i) \quad \forall i \in \{1, \dots, N\}$   
 where  $y = f(x_1, \dots, x_N)$  is the output of the computation and  $\text{View}_i$  represents the view of party  $P_i$ , including its input and the messages exchanged. We denote  $\langle \Pi_1(x_1), \dots, \Pi_N(x_N) \rangle \rightarrow y$  an execution of  $\Pi$  that outputs  $y$ .
- **Ideal World:** In the ideal world, a trusted third party computes the function  $f(x_1, x_2, \dots, x_N)$  on the inputs  $x_1, x_2, \dots, x_N$  and returns the output  $y$  to the participants without revealing any additional information:  
 $\text{IDEAL}(x_1, x_2, \dots, x_N) \rightarrow y$ .

A simulator  $\mathcal{S}$  in both models constructs a simulated execution with an adversary  $\mathcal{A}$  such that the outcome of a real execution of the protocol with  $\mathcal{A}$  has the same distribution as that of an ideal execution with  $\mathcal{S}$ . The simulator  $\mathcal{S}$  is given as inputs the outputs of the ideal-world execution and the inputs of the corrupted parties. We denote

$$\text{EXEC}_{\text{ideal}, \mathcal{S}, \mathcal{A}}(\mathcal{S}^{\mathcal{A}}(\text{IDEAL}(x_1, x_2, \dots, x_N))) \rightarrow (y, \text{View}_i) \quad \forall i \in \{1, \dots, N\}.$$

**Definition 1.** Let  $N, t, t \in \mathbb{N}$ , and  $0 < \epsilon < 1$ . An  $N$ -party protocol  $\Pi$  is said  $(T, t, \epsilon)$ -private (for some  $T < N$ ) in the semi-honest model (resp. malicious model) if there exists an algorithm  $\mathcal{S}$  such that for any adversary  $\mathcal{A}$  running in time at most  $t$  in the semi-honest model (resp. malicious model) that can compromise up to  $T$  parties, the following distributions are  $\epsilon$ -indistinguishable:  $\text{EXEC}_{\text{real}, \mathcal{A}}(x_1, x_2, \dots, x_N)$  and  $\text{EXEC}_{\text{ideal}, \mathcal{S}, \mathcal{A}}(\mathcal{S}^{\mathcal{A}}(\text{IDEAL}(x_1, x_2, \dots, x_N)))$ .

## 2.2 Threshold public-key encryption

We provide a definition of threshold PKE as the existing literature primarily offers definitions for non-interactive decryption methods (e.g. [BPR24, LY20]).

**Definition 2.** Let  $N, t \in \mathbb{N}$ . A threshold public-key encryption scheme is a triple  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  of (interactive) algorithms where:

- $\mathcal{K}(1^\lambda, N, T) \rightarrow (pk, sk_1, \dots, sk_N)$  is the probabilistic key generation algorithm. It takes as inputs the security parameter  $\lambda \in \mathbb{N}$ , the number  $N$  of decryptors and the threshold  $T$ . It outputs a public key  $pk$  (that defines a message space  $\mathcal{M}$ ), and secret keys  $sk_1, \dots, sk_N$ .
- $\mathcal{E}(pk, m) \rightarrow c$  is the probabilistic encryption algorithm. It takes as inputs the public key  $pk$  and a message  $m \in \mathcal{M}$ , and it outputs a ciphertext  $c$ .
- $\mathcal{D}(pk, c, N, T, i, sk_i)$  is an interactive probabilistic decryption algorithm. It takes as inputs the public key  $pk$ , a ciphertext  $c$ , an integer  $i \in \{1, \dots, N\}$  and a secret key  $sk_i$ . It defines an interactive protocol that outputs a message  $m \in \mathcal{M}$  or a special rejection symbol  $\perp$ .

The triple  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  should satisfy the correctness property, which ensures that an honestly generated ciphertext should be correctly decrypted by any subset of

$T$  decryptors. More formally, there exists a negligible function  $\text{negl}$  such that for all  $1 \leq T \leq N$ , all subsets  $\mathcal{G}$  of  $\{1, \dots, N\}$  of cardinal  $T$

$$\Pr \left[ m' = m \mid \begin{array}{l} (pk, sk_1, \dots, sk_N) \leftarrow \mathcal{K}(1^\lambda, N, T) \\ m \leftarrow \mathcal{M}; c \leftarrow \mathcal{E}(pk, m) \\ sk_i \leftarrow \perp, \forall i \in \{1, \dots, N\} \setminus \mathcal{G} \\ m' \leftarrow \langle \mathcal{D}(pk, c, N, t, 1, sk_1), \dots, \mathcal{D}(pk, c, N, t, N, sk_N) \rangle \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

The regular notion of public-key encryption is obtained by restricting the parameters to  $N = T = 1$  (and the decryption algorithm is then non-interactive).

For security, it is natural to consider an insider adversary allowed to corrupt some decryptors at any time and that can thus obtain the secret keys of those decryptors. One can then readily adapt the usual security notions of public-key encryption against chosen-plaintext attacks (notably one-wayness, OW-CPA, and indistinguishability IND-CPA). For security against chosen-ciphertext security, the definitions are more complex but one can follow the general security notions for MPC and replace the (non-interactive) decryption oracle by an execution of the interactive decryption protocol where the adversary is allowed to corrupt a subset of parties and participates in the decryption protocol for each decryption query. We say that a threshold public-key encryption  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  is  $(T, t, \epsilon)$ -IND-CCA2 secure (for some  $T < N$ ) in the semi-honest model (*resp.* malicious model) if it achieves the classical notion of  $\epsilon$ -IND-CCA2 security against any adversary  $\mathcal{A}$  running at most in time  $t$ , in the semi-honest model or malicious model, that can compromise up to  $T$  decryptors during the execution of the IND-CCA2 game.

We need to introduce some specific notions for the PKE that we study. We need our OW-CPA PKE (from which we build our secure cryptosystem) to be deterministic and perfectly correct. Moreover, our generic conversion is in two steps. Our intermediate PKE follows a property that we call recoverable randomness. Basically, it is possible to recover the randomness used during encryption given the secret key.

**Definition 3.** Let  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  be a public key encryption scheme with space of message  $\mathcal{M}$  and of randomness  $\mathcal{R}$ . We say that  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$ :

- is deterministic if  $\mathcal{E}$  is a deterministic algorithm.
- is perfectly correct if for every key pair  $(sk, pk)$  computed by  $\mathcal{K}$  and every message  $m \in \mathcal{M}$ ,  $\mathcal{D}(\mathcal{E}(m, pk), sk) = m$ .
- has recoverable randomness if for every key pair  $(sk, pk)$  computed by  $\mathcal{K}$ , it exists an algorithm  $\mathcal{W}$  such that for every message  $m \in \mathcal{M}$  and randomness  $r \in \mathcal{R}$ ,  $\mathcal{W}$  outputs  $r$  on input  $\mathcal{E}(m, r, pk)$  and  $sk$ .

### 2.3 Zero-knowledge proof system

We introduce a definition of non-interactive zero-knowledge (*NIZK*) proof system that is similar to the one in [Fis05]. We only consider *NIZK* proof systems that rely on hash functions modeled as random oracles.



For  $\mathcal{L} \in NP$ , we note by  $R_{\mathcal{L}}$  a deterministic function computable in polynomial time such that for all  $x \in \{0,1\}^*$ , there exists  $w$  such that  $R_{\mathcal{L}}(x, w) = 1$  if and only if  $x \in \mathcal{L}$ . If this is the case, we say that  $w$  is a witness for  $x$ .

**Definition 4.** Let  $\mathcal{L} \in NP$ ,  $t \in \mathbb{N}$ , and  $0 < \epsilon_{ZK} < 1$ . The algorithms  $(\mathcal{P}, \mathcal{V})$  are an adaptive, non-interactive  $(t, \epsilon)$ -zero-knowledge (NIZK) proof system for the language  $\mathcal{L}$ , using a list of random oracles  $G$ , if:

1. (Completeness): for  $x \in \mathcal{L} \cap \{0,1\}^\lambda$  and  $w$  such that  $R_{\mathcal{L}}(x, w) = 1$ :

$$\Pr[ \mathcal{V}^G(x, \pi) = 1 \mid \pi \leftarrow \mathcal{P}^G(x, w) ] = 1.$$

2. (Zero Knowledge): there exists an algorithm  $\mathcal{S}$  such that, for any two-staged adversary  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$  running in time at most  $t$ , the two games  $\mathbf{ZK}^{\mathcal{S}}$  and  $\mathbf{ZK}^{real}$  described below ( $st$  denotes state information between two stages) are  $\epsilon$ -indistinguishable, i.e.:

$$| \Pr[\mathbf{ZK}^{real}(\mathcal{A}) = 1] - \Pr[\mathbf{ZK}^{\mathcal{S}}(\mathcal{A}) = 1] | \leq \epsilon$$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Game <math>\mathbf{ZK}^{real}(\mathcal{A})</math>:</p> <p><math>(st, x, w) \leftarrow \mathcal{A}_1^G(1^\lambda)</math></p> <p>if <math>R_{\mathcal{L}}(x, w) = 1</math> :</p> <p style="padding-left: 20px;"><math>\pi \leftarrow \mathcal{P}^G(x, w)</math></p> <p>Else:</p> <p style="padding-left: 20px;"><math>\pi \leftarrow \perp</math></p> <p style="padding-left: 20px;"><math>b \leftarrow \mathcal{A}_2^G(st, x, \pi)</math></p> <p style="padding-left: 20px;">return <math>b</math></p> | <p>Game <math>\mathbf{ZK}^{\mathcal{S}}(\mathcal{A})</math>:</p> <p><math>(st, x, w) \leftarrow \mathcal{A}_1^G(1^\lambda)</math></p> <p>if <math>R_{\mathcal{L}}(x, w) = 1</math> :</p> <p style="padding-left: 20px;"><math>\pi \leftarrow \mathcal{S}^{G_{write}}(x)</math></p> <p>Else:</p> <p style="padding-left: 20px;"><math>\pi \leftarrow \perp</math></p> <p style="padding-left: 20px;"><math>b \leftarrow \mathcal{A}_2^G(st, x, \pi)</math></p> <p style="padding-left: 20px;">return <math>b</math></p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The notion of soundness is omitted here, as all the proof systems that we consider will follow a special case of straight-line extractability, (our notion called one-time simulation straight-line extractability). As explained before, this property implies in particular special soundness.

We will say that a proof  $\pi$  for a element  $x$  is correct (resp. incorrect) if  $\mathcal{V}^G(x, \pi) = 1$  (resp.  $\mathcal{V}^G(x, \pi) = 0$ ). We say that a statement  $x$  is true (resp. false) if  $x \in \mathcal{L}$  (resp.  $x \notin \mathcal{L}$ ). Note that  $\mathcal{P}$  might be a probabilistic algorithm. We will make an abuse of notation and denote as  $\mathcal{P}^G(x, w, r)$  the execution of  $\mathcal{P}$  using random coins  $r$ .

To lower the size of the NIZK proof, it is a standard technique [KKW18] to use a pseudorandom generator (PRG), which allows to derive most of the inputs of the MPC-in-the-head protocols from a seed for each player. These seeds are themselves derived from a tree PRG, which is a structured variant of a PRG that allows for an even shorter proof.

**Definition 5.** Let  $G : \{0,1\}^* \rightarrow \{0,1\}^*$  and  $l(\cdot)$  such that for all  $s \in \{0,1\}^\lambda$ , we have  $G(s) \in \{0,1\}^{l(\lambda)}$ . Let  $0 < \epsilon < 1$  and  $t, n \in \mathbb{N}$ . We say that  $G$  is a  $(t, \epsilon)$ -secure PRG if the following the two distributions  $G(\mathcal{U}(\{0,1\}^\lambda))$  and  $\mathcal{U}(\{0,1\}^{l(\lambda)})$  are  $(t, \epsilon)$ -indistinguishable (i.e. no adversary running in time at most  $t$  can distinguish between the two distributions with advantage more than  $\epsilon$ )

## 2.4 Existing MPC protocols

In all our MPC and MPC-in-the-head protocols, we deal with the sharing of elements in a base field that is of characteristic 2. For our MPC protocols, we consider that parties have a way to add two shared values, multiply a shared value by a public one, and multiply two shared values, in a way that is secure for either the semi-honest or malicious model. All our MPC protocols will inherit the security notions that these base operations protocols abide by. As long as the protocols for "base operations" are private in a given model, then so are our protocols. We are mainly interested in the communication complexity of our MPC protocols. In nearly all MPC primitives, that corresponds to the number of multiplications that are done between two shared elements of the field (calling them secure multiplications or shared products). All the MPC protocols use a constant number of round.

As our computation are mainly described with polynomials or matrices, we consider shared polynomials and matrices as their natural extension with arrays of shared coefficients, e.g.  $[f] = [\sum_{i=0}^d f_i X^i]$  is simply  $([f_0], \dots, [f_d])$ . We will base our computation on the following existing constant-round protocols (a more precise description is given in App. A).

- **FieldInv**: computes a shared field element  $[a^{-1}]$  given a shared non-zero field element  $[a]$  with one secure multiplication.
- **FirstPowers**: takes a non-negative integer  $k$  and a shared non-zero field element  $[b]$  and compute all the powers  $[b^2], [b^3], \dots, [b^k]$  and requires  $\mathcal{O}(k)$  secure multiplications.
- **EqTesting**: proves that a shared element  $[a]$  is zero and requires  $\mathcal{O}(1)$  secure multiplications.
- **PolyMul**: multiplies two shared polynomials of degree  $n$  in  $\mathbb{F}_q[X]$  with  $\mathcal{O}(n)$  secure multiplications in  $\mathbb{F}_q$ .
- **PolyDiv**: computes the quotient and the remainder of two shared polynomials  $[f]$  and  $[g]$  with  $\deg f \geq \deg g$  using  $\mathcal{O}(\deg f)$  secure multiplications.
- **FastEval**: computes the evaluations of a shared polynomial of  $\mathbb{F}_q[X]$  of degree less than  $d$  on a set of  $d$  shared elements of  $\mathbb{F}_q$  with  $\mathcal{O}(\tau \cdot d^{1+1/\tau})$  shared products and  $\mathcal{O}(\tau)$  rounds for any constant parameter  $\tau$ .
- **RatRecon**: given two shared polynomials  $[u]$  and  $[g]$ , computes the two shared polynomials  $[v]$  and  $[r]$  such that  $uv = r \pmod{g}$ , with degree constraints on  $r$  and  $v$  with  $\mathcal{O}(d^3)$  secure products where  $d = \deg g$ . For our need, as pointed out in [THO23], the exact degree of  $v$  and  $r$  is known in advance.

**Multiparty product verification protocol.** We now detail one of the main component that will be used in our MPCitH proof of knowledge of syndrome decoding. This technique consists in verifying that the values of a triple  $\mathbb{F}([x], [y], [z])$  of shared elements over a field follow the relation  $xy = z$ , also known as a Beaver triple [Bea92]. To verify this relation with high probability, the authors of [BN20] and [LN17] describe a protocol that "consumes" a

Beaver triple  $([a], [b], [c])$  (that can be distributed to the players at the beginning of the protocol) as well as a uniformly random value  $\epsilon \in \mathbb{F}$  (known to all the players). In our MPCitH protocol for the NIZK proof, we will need to also check that a shared triplet  $([\mathbf{X}], [\mathbf{y}], [\mathbf{z}]) \in \mathbb{F}^{D \times d} \times \mathbb{F}^d \times \mathbb{F}^D$  follows the relation  $\mathbf{X}\mathbf{y} = \mathbf{z}$ . To do this, we can generalize the verification protocol from [BN20, LN17, KZ22] to obtain a protocol that consumes a uniformly random vector  $\epsilon \in \mathbb{F}^D$  as well as a matrix-vector triple instead of a Beaver triple, i.e. a random triple  $([\mathbf{A}], [\mathbf{b}], [\mathbf{c}]) \in \mathbb{F}^{D \times d} \times \mathbb{F}^d \times \mathbb{F}^D$  such that  $\mathbf{A}\mathbf{b} = \mathbf{c}$ . The protocol, named **ProdVerif** <sub>$d, D$</sub> , takes as inputs  $([\mathbf{X}], [\mathbf{y}], [\mathbf{z}])$ ,  $([\mathbf{A}], [\mathbf{b}], [\mathbf{c}])$ , and  $\epsilon$ , and works as follows:

1. Parties locally compute  $[\alpha] \leftarrow \text{diag}(\epsilon) \cdot [\mathbf{X}] + [\mathbf{A}]$  and  $[\beta] = [\mathbf{y}] + [\mathbf{b}]$
2. Parties publish their share of  $\alpha \in \mathbb{F}^{D \times d}$  and  $\beta \in \mathbb{F}^d$
3. Parties locally compute  $[\mathbf{v}] = \text{diag}(\epsilon)[\mathbf{z}] - [\mathbf{c}] + \alpha[\mathbf{b}] + [\mathbf{A}]\beta - \alpha[\beta]$
4. Parties output their shares of  $[\alpha]$ ,  $[\beta]$ , and  $[\mathbf{v}]$ .

Note that in the case  $D = 1$  and  $d = 1$ , this protocol is exactly the one in [BN20]. At the end of the protocol,  $\mathbf{v} = 0$  if  $\mathbf{X}\mathbf{y} = \mathbf{z}$ . Moreover, for each row  $\mathbf{X}_i$  of  $\mathbf{X}$ , if  $\langle \mathbf{X}_i, \mathbf{y} \rangle \neq \mathbf{z}_i$  then there is an error with a probability of  $1/\#\mathbb{F}$  that  $\mathbf{v}_i = 0$  (i.e. an incorrect relation is accepted at row  $i$ ).

## 2.5 The Niederreiter cryptosystem

The McEliece cryptosystem [McE78] was the first code-based PKE to be introduced. In this cryptosystem, the sender would encode their message as a codeword  $c$ , using a generating matrix of the code (the public key), and add a secret noise  $e$  so that only the receiver is able to remove it using its secret key. While the code has some structure that is exploited by the receiver for decoding, the generating matrix does not reveal any information about such a structure, so that only the receiver is able to decrypt the ciphertext. In Niederreiter's variant of McEliece cryptosystem [Nie86], the sender would send only  $He$ , where  $H$  is a parity check matrix for the code used by the cryptosystem, and the message can be recovered from  $e$ . Further improvements led to the Classic McEliece Key Encapsulation Mechanism (KEM) [ABC<sup>+</sup>22], which uses an improved variant of Niederreiter's cryptosystem at its core. Notably,  $H$  is chosen as a matrix in systematic form, which allows for improvements in terms of public key size without any loss in security. All of these cryptosystems are based on binary Goppa codes. We present here the basic definition of Goppa codes as well as a modified version of Patterson's algorithm [Pat75]. For a more detailed overview of Goppa Codes and their decoding, see [Ber22].

**Definition 6.** Let  $n, m, t \in \mathbb{N}$  such that  $t$  is even. Let  $g \in \mathbb{F}_{2^m}[X]$  be a monic irreducible polynomial of degree  $t$  and  $\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_{2^m}^n$ . The Goppa code derived from  $(g, \alpha)$  is defined as  $\mathcal{C}(g, \alpha) = \{c \in \mathbb{F}_2^n \mid \sum_{i=1}^n \frac{c_i}{X - \alpha_i} = 0 \pmod{g}\}$ .

The polynomial equation defining  $\mathcal{C}(g, \alpha)$  can be seen as  $t$  equations over  $\mathbb{F}_{2^m}$ , or  $t \cdot m$  equations over  $\mathbb{F}_2$ . Therefore, the code has dimension at least

$k = n - mt$ . Goppa codes can be decoded efficiently if the parameters  $(g, \alpha)$  are known, and in code-based cryptosystems, the decoding algorithm is a crucial step in decryption. One algorithm for the decoding of binary Goppa codes is the Patterson algorithm [Pat75]. It takes the parameters  $(g, \alpha)$  of a Goppa code as well as a word  $y \in \{0, 1\}^n$  as input. If there exists  $e \in \{0, 1\}^n$  such that  $\text{wt}(e) \leq t$  and  $y - e$  is a codeword of the Goppa code defined by  $(g, \alpha)$ , then the algorithm outputs  $e$  (i.e. the Patterson algorithm outputs the error corresponding to the input). An MPC protocol that performs the same operation as this algorithm is crucial for our construction. Algorithm 1 is a version of the Patterson algorithm similar to the one in [THO23]. It only deals with the cases for which  $t$  is even and  $\text{wt}(e) = t$ , but this is enough for our use case and is easier to study for the design of an MPC protocol.

---

**Algorithm 1: Decoding**

---

**Parameters:**  $n, m, t \in \mathbb{N}$  such that  $t$  is even.  
**Input:**  $g \in \mathbb{F}_{2^m}[X]$  irreducible polynomial of degree  $t$ ,  
 $\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_{2^m}^n$ ,  $y \in \mathbb{F}_2^n$  such that there exist  $c \in \mathcal{C}(g, \alpha)$  and  $e$   
such that  $\text{wt}(e) = t$  and  $y = c + e$   
**Output:**  $e \in \mathbb{F}_2^n$

```

1 for  $i$  from 1 to  $n$  do
2   | Compute  $(X - \alpha_i)^{-1} \bmod g$ 
3 end
4  $s \leftarrow \sum_{i=1}^n y_i (X - \alpha_i)^{-1} \bmod g$ 
5  $u \leftarrow (s^{-1} + X)^{1/2} \bmod g$  ▷ computed as element of  $\mathbb{F}_{2^m}[X]/g$ 
6 Compute  $r$  and  $v$  such that  $uv = r \bmod g$  s.t.  $\deg(r) = t/2$ , and
    $\deg(v) = t/2 - 1$  ▷ rational reconstruction
7  $\epsilon \leftarrow r^2 + Xv^2$ 
8  $e \leftarrow (0, \dots, 0)$ 
9 for  $i$  from 1 to  $n$  do
10  | if  $\epsilon(\alpha_i) = 0$  then
11    |  $e_i \leftarrow 1$ 
12  | end
13 end
14 Return  $e = (e_1, \dots, e_n)$ 

```

---

In the Classic McEliece cryptosystem [ABC<sup>+</sup>22], which is a IND-CCA key encapsulation mechanism (KEM), the core component is a OW-CPA PKE for which the secret key is the parameters  $(g, \alpha)$  of a secret Goppa code and the public key is a parity matrix  $H$  with a systematic form that is  $(I_{n-k}|T)$  where  $k = n - mt$ . Its plaintext space is  $\{e \in \{0, 1\}^n | \text{wt}(e) = t\}$ . For our use case, we will start from the improved version of Niederreiter's cryptosystem used in the Classic McEliece KEM. This PKE is deterministic, perfectly correct and OW-CPA. We denote this PKE by  $(\mathcal{K}^{ME}, \mathcal{E}^{ME}, \mathcal{D}^{ME})$  and we describe it in figure 1.

- $\mathcal{K}^{ME}(1^\lambda)$ : Deduce the parameters  $(n, m, t)$  from  $\lambda$ . Generate a random vector  $\alpha \in \mathbb{F}_{2^m}^n$  and a random monic irreducible  $g \in \mathbb{F}_{2^m}[X]$  of degree  $t$ . Compute the parity check matrix  $H$  of  $\mathcal{C}(g, \alpha)$  in systematic form. If  $H$  is not of the form  $(I_{n-k}|T)$ , retry with different  $(g, \alpha)$ . Return  $pk = T$  and  $sk = (g, \alpha, T)$ .
- $\mathcal{E}^{ME}(e, pk)$ : Compute  $H = (I_{n-k}|T)$ . Return  $c = He$ .
- $\mathcal{D}^{ME}(c, sk)$ : Compute  $y = (c|0, \dots, 0)$  by appending  $c$  by  $k$  zeroes. Set  $(g, \alpha, T) = sk$  and run the algorithm 1 on inputs  $(g, \alpha, y)$  to obtain  $e$ . If  $He = c$  and  $\text{wt}(e) = t$ , return  $e$ . Otherwise, return  $\perp$ .

**Fig. 1.** The Niederreiter cryptosystem as described in the Classic McEliece submission

### 3 A generic conversion from an OW-CPA to an IND-CCA2 cryptosystem

The first step that we take to design an IND-CCA threshold scheme is to convert the Niederreiter cryptosystem (which is OW-CPA) into an IND-CCA non-threshold scheme. In this section, we present a generic conversion inspired by the Naor Yung conversion [NY90] to upgrade the security of an encryption scheme from OW-CPA to IND-CCA2. This conversion is applied to the Niederreiter cryptosystem in section 4.

To design our generic conversion, our approach has two steps: The first one is a classical conversion from a OW-CPA secure cryptosystem  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$ , that uses a hash function modeled as a random oracle. It consists on using the OW-CPA system to encrypt a random element  $r$  whose hashed value is used as a one-time pad on the plaintext. Both the encryption of  $r$  and the padded message are sent (see Fig. 2). The second step adds a non-interactive proof that the previous ciphertext is well-formed, namely a proof system for the following language :

$$\mathcal{L} = \{(pk, c) \mid \exists (m, r) \in \mathcal{M} \times \mathcal{R}, c = \mathcal{E}(m, r, pk)\}.$$

This non-interactive proof must abide by a specific security notion, that we call *(one-time) simulation straight-line extractability* in the random oracle model. Most NIZK proof systems achieve special soundness, which guarantees the existence of an extractor that can extract the witness of a language from an adversary that is capable of producing accepting proofs with a good probability, with a rewindable black-box access to this adversary. This suffices to achieve non-adaptative chosen-ciphertext (i.e. IND-CCA1) security. Sahai [Sah99] showed that if the proof system is non-malleable, then the conversion from Naor and Yung can be made secure against adaptively chosen-ciphertext (i.e. IND-CCA2) attacks. Lindell [Lin03b] proved a similar result where the NIZK proof systems need to be (one-time) simulation sound.

Moreover, the constructions in [NY90], [Sah99] and [Lin03b] need to encrypt the same plaintext twice under two different keys. In order to have to encrypt the plaintext only once, the NIZK proofs used in the conversion need to be straight-line extractable. This notion means that there exists an extractor which can recover a witness of a proof only by looking at the queries of the adversary

to the oracles, instead of having full control over its execution (as it is the case for special soundness).

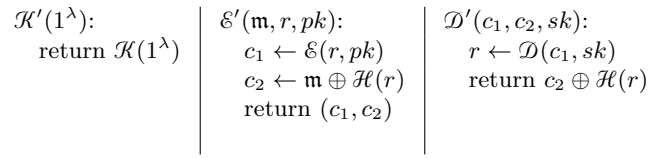
To prove the security of our construction (see Fig. 4), we need a property that combines both simulation soundness from [Lin03b] and straight-line extraction from [Fis05]. It is a special flavor of straight-line extractability, in which the adversary can see a simulated proof of its choice before constructing a proof. We formally describe our property in the following definition 7.

**Definition 7.** Let  $\mathcal{L} \in NP$ ,  $t_{ZK}, t_s \in \mathbb{N}$ ,  $\epsilon_{ZK}, \epsilon_s \in ]0, 1[$ , and let  $(\mathcal{P}, \mathcal{V})$  be a  $(t_{ZK}, \epsilon_{ZK})$ -NIZK proof system for  $\mathcal{L}$  using a list of random oracles  $G$ . Let  $\mathcal{S}$  be a simulator for  $(\mathcal{P}, \mathcal{V})$ . We say that an algorithm  $\mathcal{T}$  is a  $(t_s, \epsilon_s)$ -one-time simulation extractor for  $(\mathcal{P}, \mathcal{V})$ , or that  $(\mathcal{P}, \mathcal{V})$  is  $(t_s, \epsilon_s)$ -one-time simulation straight-line extractable with extractor  $\mathcal{T}$ , if every adversary  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$  running in time at most  $t_s$  wins the game described below with probability less than  $\epsilon_s$ . We denote by  $\mathcal{Q}(\mathcal{A}^G)$  the list of queries from  $\mathcal{A}$  to each of the random oracles from  $G$  and the answers of each random oracles from  $G$ .

Game **Exp**<sup>simextract</sup>( $\mathcal{A}$ ):

$(st, x) \leftarrow \mathcal{A}_1^G(1^\lambda)$  (the adversary chooses a statement for a simulated proof)  
 $\pi \leftarrow \mathcal{S}^{G_{write}}(x)$   
 $(x', \pi') \leftarrow \mathcal{A}_2^G(st, x, \pi)$  (the adversary tries to output a proof)  
 $w' \leftarrow \mathcal{T}^G(x', \pi', \mathcal{Q}(\mathcal{A}^G))$   
 $\mathcal{A}$  wins if and only if  $R_{\mathcal{L}}(x', w') = 0$ ,  $(x, \pi) \neq (x', \pi')$ , and  $\mathcal{V}^G(x', \pi') = 1$ .

We begin by describing the first step of our conversion: It generates a random element to be encrypted under the OW-CPA scheme, and then uses a hash function  $\mathcal{H} : \mathcal{M} \rightarrow \{0, 1\}^l$  to completely hide the plaintext. This leads to an IND-CPA scheme  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$ . This is a folklore result, which, for completeness, is stated here for any deterministic and perfectly correct OW-CPA scheme  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$ . Figure 2 and theorem 1 follow.



**Fig. 2.** Generic conversion from OW-CPA to IND-CPA security

**Theorem 1.** Let  $0 < \epsilon < 1$ , and let  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  be a public key encryption scheme with message space  $\mathcal{M}$  and of ciphertext space  $\mathcal{C}$ , which is deterministic and perfectly correct. Let  $\mathcal{H} : \mathcal{M} \rightarrow \{0, 1\}^l$  be a random oracle. Then the PKE scheme denoted  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$  described in figure 2, with message space  $\{0, 1\}^l$  and randomness space  $\mathcal{M}$ , is  $(t, \epsilon)$ -IND-CPA if  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  is  $(t', \epsilon)$ -OW-CPA, with

$$t' = t + \tau_{\mathcal{E}} \cdot q_{\mathcal{H}},$$

where  $q_{\mathcal{H}}$  is an upper bound on the number of queries made by an OW-CPA adversary running in time  $t$ , and  $\tau_{\mathcal{E}}$  is the running time of  $\mathcal{E}$ . Moreover,  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$  has recoverable randomness (with the randomness extractor running in the same time as  $\mathcal{D}$ ).

We now describe the second step of the conversion, that uses an IND-CPA scheme  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  with recoverable randomness, and adds a NIZK proof that the ciphertext is well built. We prove that if the NIZK proof system is (one-time) simulation straight-line extractable, then the resulting abide by a stronger notion of IND-CCA2 security, that we call IND-CCA2<sup>+</sup>. This notion is specific to our conversion, and it will only be helpful in order to prove the security of our threshold cryptosystem in section 5. Since our MPC protocol will not give the resulting ciphertext but will also reveal the randomness associated to the ciphertext, we need to ensure that revealing this randomness will not compromise the security of the resulting PKE. In our notion called IND-CCA2<sup>+</sup>, the attacker has access to a decryption oracle that also gives the randomness associated to the ciphertext. We define this notion formally in definition 8. We present the generic conversion in figure 3. Then, we state in theorem 2 that our generic conversion indeed follows IND-CCA2<sup>+</sup> security. Its proof can be found in App. B.

$$\begin{array}{c|c|c}
 \mathcal{K}'(1^\lambda): & \mathcal{E}'(\mathbf{m}, (r, r'), pk): & \mathcal{D}'((c, \pi), (pk, sk)): \\
 (pk, sk) \leftarrow \mathcal{K}(1^\lambda) & c \leftarrow \mathcal{E}(\mathbf{m}, r, pk) & \text{If } \mathcal{V}^G((pk, c), \pi) = 1: \\
 \text{return } (pk, (pk, sk)) & \pi \leftarrow \mathcal{P}^G((pk, c), (\mathbf{m}, r), r') & \text{return } \mathcal{D}(c, sk) \\
 & \text{return } (c, \pi) & \text{Else:} \\
 & & \text{return } \perp
 \end{array}$$

**Fig. 3.** Generic conversion from IND-CPA to IND-CCA2<sup>+</sup> security

**Definition 8.** Let  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  be a PKE, and  $(\mathcal{P}, \mathcal{V})$  be a NIZK proof system using a list of random oracles  $G$ . Let  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$  be the resulting PKE from the conversion from figure 3. We denote by  $O(\cdot, pk, sk)$  an oracle that, given a ciphertext  $(c, \pi)$  for  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$ , returns  $(\mathbf{m}, r)$ , where  $c = \mathcal{E}(\mathbf{m}, r, pk)$  if  $\mathcal{V}^G((pk, c), \pi) = 1$ , and returns  $\perp$  otherwise. We say that  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$  is  $(t, \epsilon)$ -IND-CCA2<sup>+</sup> if for any two-staged adversary  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$  running in time at most  $t$ , we have the following:

$$|\Pr[\mathbf{Exp}^{IND-CCA2^+}(\mathcal{A}, 0) = 1] - \Pr[\mathbf{Exp}^{IND-CCA2^+}(\mathcal{A}, 1) = 1]| \leq \epsilon$$

where  $\mathbf{Exp}^{IND-CCA2^+}$  is defined as such:

$$\begin{array}{l}
 \text{Game } \mathbf{Exp}^{IND-CCA2^+}(\mathcal{A}, b): \\
 (pk, sk) \leftarrow \mathcal{K}(1^\lambda) \\
 (state, \mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}^{O(\cdot, pk, sk), G}(pk) \\
 c^* \leftarrow \mathcal{E}(\mathbf{m}_b, pk) \\
 b^* \leftarrow \mathcal{A}^{O(\cdot, pk, sk), G}(state, c, pk) \\
 \text{return } b^*
 \end{array}$$

In definition 8, the oracle  $O$  acts as the decryption oracle and gives the randomness corresponding to the ciphertext as well. In the proof in Appendix B, we will refer to this oracle as the decryption oracle with randomness.

**Theorem 2.** *Let  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  be a public key encryption scheme with message space  $\mathcal{M}$  and randomness space  $\mathcal{R}$ , and that has recoverable randomness. Let  $(\mathcal{P}, \mathcal{V})$  be a NIZK using a list of random oracles  $G$  for the following language:*

$$\mathcal{L} = \{(pk, c) \mid \exists (\mathbf{m}, r) \in \mathcal{M} \times \mathcal{R}, c = \mathcal{E}(\mathbf{m}, r, pk)\}.$$

*Furthermore, we denote  $\epsilon$  as  $\epsilon = \epsilon_{cpa} + 2\epsilon_{ZK} + 2\epsilon_s$ . Then, the public key encryption scheme  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$  described in figure 3 is  $(t, \epsilon)$ -IND-CCA2<sup>+</sup> if  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  is  $(t_{cpa}, \epsilon_{cpa})$ -IND-CPA,  $(\mathcal{P}, \mathcal{V})$  is  $(t_{ZK}, \epsilon_{ZK})$ -zero-knowledge, and  $(t_s, \epsilon_s)$ -one-time simulation straight-line extractable, such that:*

$$\begin{aligned} t_{ZK} &= t + \tau_{\mathcal{K}} + \tau_{\mathcal{E}} + q_O(\tau_{\mathcal{V}} + \tau_{\mathcal{D}} + \tau_{rand}), \\ t_{cpa} &= t + \tau_{\mathcal{S}} + q_O(\tau_{\mathcal{V}} + \tau_{\mathcal{J}}), \\ t_s &= t + \tau_{\mathcal{K}} + \tau_{\mathcal{E}} + q_O(\tau_{\mathcal{V}} + \tau_{\mathcal{J}} + \tau_{\mathcal{E}}), \end{aligned}$$

where  $\tau_{\mathcal{K}}, \tau_{\mathcal{E}}, \tau_{\mathcal{D}}$  are the running times of  $\mathcal{K}, \mathcal{E}$ , and  $\mathcal{D}$  respectively,  $\tau_{rand}$  is the running time of the randomness extractor of  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$ ,  $\tau_{\mathcal{S}}$  is the running time for the simulator for the ZK property of  $(\mathcal{P}, \mathcal{V})$ ,  $\tau_{\mathcal{J}}$  is the running time of the extractor for the straight-line extractability of  $(\mathcal{P}, \mathcal{V})$ , and  $q_O$  is an upper bound for the number of queries made by an adversary running in time at most  $t$  to the decryption oracle with randomness.

The two intermediate conversions described in Fig. 2 and 3 can be combined to obtain the full conversion that we aim for, which is described in Fig. 4. This conversion builds an IND-CCA2 scheme  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$  from an OW-CPA scheme  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$ . The resulting IND-CCA2 scheme (see Corollary 1) has the particularity that one can publicly check if a ciphertext is correctly formed thanks to the NIZK proof.

|                                                                                                       |                                                                                                                                                                                                                               |                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\mathcal{K}'(1^\lambda):$<br>$(pk, sk) \leftarrow \mathcal{K}(1^\lambda)$<br>return $(pk, (pk, sk))$ | $\mathcal{E}'(\mathbf{m}, r, pk):$<br>$c_1 \leftarrow \mathcal{E}(r, pk)$<br>$c_2 \leftarrow \mathbf{m} \oplus \mathcal{H}(r)$<br>$\pi \leftarrow \mathcal{P}^G((pk, c_1, c_2), (\mathbf{m}, r))$<br>return $(c_1, c_2, \pi)$ | $\mathcal{D}'((c_1, c_2, \pi), (pk, sk)):$<br>If $\mathcal{V}^G((pk, c_1, c_2), \pi) = 1:$<br>$r \leftarrow \mathcal{D}(c_1, sk)$<br>return $c_2 \oplus \mathcal{H}(r)$<br>Else:<br>return $\perp$ |
|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 4.** Generic conversion from OW-CPA to IND-CCA2<sup>+</sup> security

**Corollary 1.** *Let  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  be a deterministic and perfectly correct public key encryption scheme with message space  $\mathcal{M}$ , and  $\mathcal{H} : \mathcal{M} \rightarrow \{0, 1\}^l$  a random*



oracle. Let  $(\mathcal{P}, \mathcal{V})$  be a NIZK using a list of random oracles  $G$  for the following language:

$$\begin{aligned}\mathcal{L} &= \{(pk, c_1, c_2) \mid \exists w \in \mathcal{M}, c_1 = \mathcal{E}(w, pk)\} \\ &= \{(pk, c_1, c_2) \mid \exists (w, \mathbf{m}) \in \mathcal{M} \times \{0, 1\}^l, c_1 = \mathcal{E}(w, pk), c_2 = \mathbf{m} \oplus \mathcal{H}(w)\}\end{aligned}$$

Then, the public key encryption scheme  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$  described in figure 4, with space of message  $\{0, 1\}^l$  and space of randomness  $\mathcal{M}$ , is  $(t, \epsilon)$ -IND-CCA2<sup>+</sup>, supposing that  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  is  $(t_{ow}, \epsilon_{ow})$ -OW-CPA, that  $(\mathcal{P}, \mathcal{V})$  is  $(t_{ZK}, \epsilon_{ZK})$ -zero-knowledge, and  $(t_s, \epsilon_s)$ -one-time simulation straight-line extractable, where:

$$\begin{aligned}\epsilon &= \epsilon_{ow} + 2\epsilon_{ZK} + 2\epsilon_s, \\ t_{ow} &= t + \tau_S + q_O(\tau_V + \tau_{\mathcal{T}}) + q_{\mathcal{H}}\tau_{\mathcal{E}}, \\ t_{ZK} &= t + \tau_{\mathcal{K}} + \tau_{\mathcal{E}} + q_O(\tau_V + 2\tau_{\mathcal{D}}) + q_{\mathcal{H}}\tau_{\mathcal{E}}, \\ t_s &= t + \tau_{\mathcal{K}} + \tau_{\mathcal{E}} + q_O(\tau_V + \tau_{\mathcal{T}} + \tau_{\mathcal{E}}) + q_{\mathcal{H}}\tau_{\mathcal{E}},\end{aligned}$$

where  $\tau_{\mathcal{K}}, \tau_{\mathcal{E}}, \tau_{\mathcal{D}}$  are the running times of  $\mathcal{K}, \mathcal{E}$ , and  $\mathcal{D}$  respectively,  $\tau_S$  is the running time for the simulator for the ZK property of  $(\mathcal{P}, \mathcal{V})$ ,  $\tau_{\mathcal{T}}$  is the running time of the extractor for the straight-line extractability of  $(\mathcal{P}, \mathcal{V})$ , and  $q_O$  (resp.  $q_{\mathcal{H}}$ ) is an upper bound for the number of queries made by an adversary running in time at most  $t$  to the decryption oracle with randomness (resp. to the random oracle  $\mathcal{H}$ ).

#### 4 A one-time simulation straight-line extractable NIZK for Niederreiter encryption

To instantiate our generic conversion of section 3, we need a proof system that proves the knowledge of a word of correct Hamming weight that corresponds to the attached ciphertext. As required by Corollary 1, the proof system needs to be one-time simulation straight-line extractable (see definition 7). This section describes a NIZK proof system, based on the MPC-in-the-Head (MPCitH) paradigm, that enables simulation straight-line extractability, hence allowing us to derive an IND-CCA2 variant of the Niederreiter cryptosystem.

Our NIZK proof system considers the following language, which corresponds exactly to a correctly formed ciphertext for our IND-CCA2 Niederreiter system:

$$\mathcal{L} = \{(H, c_1, c_2) \in \mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k} \times \{0, 1\}^l \mid \exists r \in \mathbb{F}_2^n, \mathbf{wt}(r) = t, c_1 = Hr\}.$$

It is worth noting that, even if  $c_2$  does not appear explicitly in the conditions that define the language  $\mathcal{L}$  (except the fact that it belongs to  $\{0, 1\}^l$ ), it is of utmost importance to include it in the language for security in the statement of Theorem 2. Indeed, a valid proof for a ciphertext  $(c_1, c_2)$  should not be used to build a valid proof for another ciphertext of the form  $(c_1, c'_2)$  (with  $c'_2 \neq c_2$ ), even though the property verified by the proof is only dependent on  $c_1$ .

Our proof system is a variant of the proof system from Feneuil, Joux and Rivain [FJR22] that is mainly designed as a signature scheme based on the hardness of a split variant of the *Syndrome Decoding* problem. To build their proof of knowledge system, they rely on a MPCitH protocol simulated by the builder of the proof. Essentially, the proof consists of all the views of the simulated players of the MPCitH protocol, except one (this way, the proof system is zero-knowledge). If  $N$  players are simulated, and the player whose view is not revealed is chosen at random, then the probability for the prover to cheat on the views of the players is bounded by  $1/N$ . To lower down this probability even more, the protocol is repeated over  $\tau$  iterations.

#### 4.1 Proof of knowledge for syndrome decoding solution

Given a matrix  $H \in \mathbb{F}_2^{(n-k) \times n}$  and a vector  $c_1 \in \mathbb{F}_2^{n-k}$ , the proof system of [FJR22] aims at proving the knowledge of  $r \in \mathbb{F}_2^n$  that satisfies the split *Syndrome Decoding* problem, i.e. that if we write  $r$  as  $r = (r_1 | \dots | r_d)$ , we have  $Hr = c_1$  and  $\text{wt}(r_i) \leq t/d$  for  $1 \leq i \leq d$ . In order to verify these  $d$  inequalities, the idea of the authors [FJR22] is to see these inequalities as  $d$  polynomial equations. Note that the idea of using polynomial equations to prove Hamming weight was already present in [DLO<sup>+</sup>18]. The polynomials are initially shared between the players, and the aim of the MPCitH protocol is to verify the equations corresponding to the  $d$  inequalities on the Hamming weight on the chunks of  $r$ . The verification of the equation  $Hr = c_1$  is not necessary as the polynomial equations are built so that  $r$  is indeed a solution of the linear system.

Our observation is the following: their proof system can be adapted to verify other polynomial equations, and therefore other inequalities on Hamming weights. In our case, instead of verifying  $\text{wt}(r_i) \leq t/d$  for  $1 \leq i \leq d$ , we want to verify  $\text{wt}(r) = t$ , which is equivalent to  $\text{wt}(r) \leq t$  and  $\text{wt}(r) \geq t$ .

**Proving small Hamming weight.** To prove  $\text{wt}(r) \leq t$ , the authors of [FJR22] propose to interpolate the unique polynomial  $S_r$  of degree  $n - 1$  such that  $S_r(\gamma_i) = r_i$ , where  $\gamma_1, \dots, \gamma_n$  are distinct public points from a finite field  $\mathbb{F}_{poly} = \mathbb{F}_{2^\ell}$  such that  $2^\ell \geq n > 2^{\ell-1}$ . By definition of  $S_r$ , it has a common factor with the public polynomial  $F = \prod_{i=1}^n (X - \gamma_i)$ . This common factor enables to verify that the Hamming weight of  $r$  is at most  $t$ , i.e.  $S_r$  has at most  $t$  common roots with  $F$  if and only if  $r$  is of Hamming weight at most  $t$ . In [FJR22], this translates to the existence of common multiple of  $F$  and  $S_r$  of degree at most  $n + t - 1$ , i.e.  $QS_r = PF$  where  $Q = \prod_{i \in [n]: r_i \neq 0} (X - \gamma_i)$ .

**Proving large Hamming weight.** To find a similar polynomial equation for large Hamming weight, a first attempt would consist of finding a similar relation as above by replacing  $S_r$  by  $(1 - S_r)$ . Indeed,  $(1 - S_r)$  and  $F$  have a common factor of degree at most  $n - t$ . This solution would lead to a polynomial relation for which the polynomials that are introduced would have degrees that are considerably higher than the ones in the first polynomial relation (i.e.  $n - t$  instead

of  $t$ ). Therefore, this solution would prove quite inefficient. We therefore need to find a polynomial relation that introduces polynomials of degrees approximately equal to  $t$ . To this end, we can remark another relation between  $F$  and  $S_r$  that also allows to prove that  $\text{wt}(r) \geq t$ . Indeed, we know that  $S_r$  and  $F$  have a common factor of degrees at most  $n - t$  if and only if  $r$  is of Hamming weight greater or equal than  $t$ . This is equivalent to saying that there exist two polynomials  $U, V$  that give a Bézout relation with  $S_r$  and  $F$  yielding a polynomial of degree at most  $n - t$ , i.e.  $\deg(US_r + VF) \leq n - t$  (since the gcd of  $S_r$  and  $F$  is of degree  $\leq n - t$ ). Since this common factor is of a known degree, i.e.  $n - t$ , we can bound the degree of the Bézout coefficients in this relation, yielding  $\deg U < t$  and  $\deg V < t - 1$  (see for instance [GG03, corollary 6.48]). Using this Bézout relation, it is enough to verify that  $US_r + VF \text{ quo } X^{n-d} = 1$  to demonstrate the existence of a common factor of  $F$  and  $S_r$  of degree at most  $n - k$ .

The following lemma establishes the necessary conditions to assert  $\text{wt}(r) = t$  with two polynomial equations with  $F$  and  $S_r$ . Due to lack of space, the proof of this lemma is deferred to appendix C.

**Lemma 1.** *Let  $\mathbb{F}_{poly}$  be a finite extension of  $\mathbb{F}_2$  such that  $|\mathbb{F}_{poly}| \geq n$ . Let  $\gamma_1, \dots, \gamma_n$  be  $n$  distinct points of  $\mathbb{F}_{poly}$ . Let  $r = (r_1, \dots, r_n) \in \mathbb{F}_2^n$  and let  $S_r \in \mathbb{F}_{poly}[X]$  be the unique polynomial of degree  $\leq n - 1$  such that  $S_r(\gamma_i) = r_i$  for  $1 \leq i \leq n$ . Let  $F = \prod_i (X - \gamma_i)$ . Then  $\text{wt}(r) = t$  if and only if there exists  $Q \in \mathbb{F}_{poly}[X]$  monic of degree  $t$ ,  $P \in \mathbb{F}_{poly}[X]$  of degree  $\leq t - 1$ , and  $U, V \in \mathbb{F}_{poly}[X]$  of degrees  $< t$  and  $< t - 1$  respectively such that:*

$$S_r Q = FP \text{ and } (S_r U + FV) \text{ quo } X^{n-t} = 1. \quad (1)$$

**Encoding the proof.** As in [FJR22], our zero knowledge MPCitH proof will encode the validity of these two equations by providing their evaluations on several points  $(\rho_j)_{j \in [1, s]}$ , that are sampled randomly as part of the challenge. We shall mention that the  $\gamma_i$ 's (and  $F$ ) will be public parameters of the proof system, and they can be re-used for several proofs. The polynomials  $S_r$ ,  $Q$  and  $P$  will be shared across all the players (in the MPCitH protocol), implying that the polynomials  $Q$  and  $P$  must be added to the proof while it is not necessary for  $S_r$ . Indeed, since the  $\gamma_i$ 's are public, each player can locally compute a share of  $S_r$  from their share of the vector  $r$ .

Following the idea of [FJR22], we can verify the first equation on all points, i.e.  $S_r(\rho_j)Q(\rho_j) = F(\rho_j)P(\rho_j)$ , by using a multiparty product verification protocol. Indeed, since the polynomial  $F$  is a public parameter and all the  $\rho_j$ 's are part of the challenge, each player can compute locally their shares of  $[c] = F(\rho_j)[P(\rho_j)]$ ,  $[a] = [S_r(\rho_j)]$  and  $[b] = [Q(\rho_j)]$ . It is then sufficient to use the protocol **ProdVerif** <sub>$d, D$</sub>  (with  $d = 1$  and  $D = 1$ ) given in section 2.4 with an appropriate Beaver triple to publish a proof of the equality.

For the second equation, i.e.  $(S_r U + FV) \text{ quo } X^{n-t} = 1$ , it is not sufficient to verify a single shared product for each evaluation point. Indeed, the presence of the quotient operator implies seeing each evaluation of the equation as a dot

product. Let us rewrite the equation as

$$S_r U \text{ quo } X^{n-t} = (FV \text{ quo } X^{n-t}) + 1, \quad (2)$$

and see that, if we denote  $U = \sum_{i=0}^{t-1} u_i X^i$  and  $V = \sum_{i=0}^{t-1} v_i X^i$ , we have  $(S_r U \text{ quo } X^{n-t})(\rho_j) = \sum_{i=0}^{t-1} u_i (S_r \text{ quo } X^{n-t-i})(\rho_j)$  and  $(FV \text{ quo } X^{n-t})(\rho_j) = \sum_{i=0}^{t-1} v_i (F \text{ quo } X^{n-t-i})(\rho_j)$ .

Here again, remarking that the polynomial  $F$  is a public parameter and all the  $\rho_j$ 's are part of the challenge, each player can locally compute: the shared vector  $[z] = [(FV \text{ quo } X^{n-t})(\rho_1), \dots, (FV \text{ quo } X^{n-t})(\rho_s)]$  of  $(\mathbb{F}_{points})^s$ ; the shared vector  $[y] = [u_0, \dots, u_{t-1}]$  belonging to  $(\mathbb{F}_{points})^t$ ; and the shared matrix  $[X] = [((S_r \text{ quo } X^{n-t-i})(\rho_j))_{i \in [0, t-1], j \in [1, s]}]$  of  $(\mathbb{F}_{points})^{s \times t}$ . Observe that the equation 2 is correct if and only if the matrix-vector product equation  $[z] + \mathbf{1} = [X][y]$  is also verified. Thus, it is sufficient to use the protocol **ProdVerif** <sub>$d, D$</sub>  (with  $d = t$  and  $D = s$ ) given in section 2.4 with an appropriate "matrix-vector" Beaver triple to publish a proof of the equation 2.

All in all, per iteration, the protocol **ProdVerif** <sub>$d, D$</sub>  is used with  $d = 1$  and  $D = 1$   $s$  times to verify the polynomial equation corresponding to  $P$  and  $Q$ , and the protocol **ProdVerif** <sub>$d, D$</sub>  is used one time with  $d = t$  and  $D = s$  to verify the polynomial equation corresponding to  $U$  and  $V$ .

**Soundness of the proof.** There are two ways to accept an incorrect proof. Either, an evaluation point  $\rho_j$  is a root of at least one of the equations of Lemma 1, or one of the call to **ProdVerif** <sub>$d, D$</sub>  on the evaluated equations at  $\rho_j$  fails, and this must happen for all the possible  $\rho_j$ 's. As in [FJR22], we will sample the points  $(\rho_j)_{j \in [1, s]}$  in a larger field  $\mathbb{F}_{points}$ , such that  $\mathbb{F}_2 \subseteq \mathbb{F}_{poly} \subseteq \mathbb{F}_{points}$ , in order to lower down the probability of error. The lemma 4 provided in Appendix E establishes the probability  $p \leq \left( \frac{n+t}{|\mathbb{F}_{point}|} \right)^s$  that an incorrect witness is formed at a given repetition of the protocol.

**Reducing the proof size.** We use the same technique as in [FJR22], following the work of [KKW18], that use a pseudo-random generator (PRG), to derive most of the inputs of the MPC-in-the-head protocols from a seed for each player. Indeed, it is always assumed that the additive sharing is ensured by the player  $N$  while the other players are given their values at random from a given seed. All these seeds are themselves derived from a tree PRG, which is a structured variant of a PRG that allows for an even shorter proof, i.e. only  $\log N$  seeds are needed instead of  $N - 1$ .

To lower down the size even more, the authors of [FJR22] impose that the parity check matrix  $H$  is given in standard form, i.e.  $H = (I_{n-k} | T)$  (this is not a constraint here as Niederreiter public keys are of this form). Indeed by splitting  $r$  as  $r = (r_B | r_A)$  where  $r_B$  and  $r_A$  are of length  $n - k$  and  $k$  respectively, we have that  $r_B = y - Tr_A$ . Since  $y$  and  $T$  are public in the MPCitH protocol, all the players just have to locally compute a share of  $r_B$  from their share of  $r_A$ , and obtain a complete share of  $r = (r_B | r_A)$ . The proof must only contain the sharing

of  $r_B$ , and the correctness of  $Hr = y$  is directly ensured by the construction of the vector  $r_B$ . Note also that since  $Q$  is a monic polynomial of a fixed degree, its leading coefficient (always equal to 1) is not shared during the MPC-in-the-head protocol.

## 4.2 Our NIZK proof system

**Constructing the proof.** We propose in Figure 5 the complete description of the protocol that constructs the zero-knowledge proof of a syndrome decoding solution, i.e.  $y = Hr$  and  $\text{wt}(r) = t$ . We emphasize that our MPCitH protocol is a 5-round protocol that is turned into a non-interactive protocol via the Fiat-Shamir transform (as in [FJR22]): In round 1.0, all the polynomials  $S_r, Q, P, U$  and  $V$  are computed and then distributed among all the players in round 1.1 together with the vector  $r_A$ . The triples needed for the multiparty product verification, i.e.  $c_j = a_j b_j$  and  $\mathbf{c}' = \mathbf{A}' \mathbf{b}'$  are also distributed among the players in round 1.1.

As in [FJR22], the prover first commits on all the initial states using a hash function  $\mathcal{H}_0$ . Then, in round 2, the prover uses the commitments to generate the random evaluation points  $\rho_j$  as well as the random values  $\epsilon$  and  $\epsilon'$  used for the verification with  $\mathbf{ProdVerif}_{d,D}$ . The first challenge  $h_1$  in the proof is constructed upon the  $\rho_j$ 's,  $\epsilon$  and  $\epsilon'$ . This is done using a hash function  $\mathcal{H}_1$ . In round 3, the MPCitH protocol for verification is computed. The result of the protocol is used in round 4 to compute the player whose view is not revealed (the index of this player is denoted by  $i^*$ ). This is the second challenge  $h_2$  of the proof that is computed through  $\mathcal{H}_2$ . Note that, contrary to the NIZK proof system in [FJR22], we include the whole statement  $(H, c_1, c_2)$  in the hash inputs for the challenges (instead of  $c_2$  only). This is done to eliminate various malleability attacks, as this ensures that the challenge is uniquely bound to the specific witness. This prevents adversarial modifications that could lead to forgery or undesirable re-randomization of proofs. The constructed proof  $\pi$  uses  $\tau$  repetitions of these five rounds to ensure a good soundness. At the end,  $\pi$  is then constituted of one salt (used in the hash function for security reasons, as in [FJR22]), the  $\tau$  challenges in  $h_1$  and  $h_2$ , and the  $\tau$  relevant information of the chosen player  $i^*$  that ensure the verification can be computed without their help. In particular, this contains their commitment  $\text{com}_{i^*}$  on all their shared initial values, and their shared values of the revealed data in the multiparty verification product protocol  $\mathbf{ProdVerif}_{d,D}$ . The proof also contains, for each of the  $\tau$  repetitions, all the states of all players except  $i^*$ , in order to compute the first challenge without the player  $i^*$ .

**Verifying the proof.** To verify a proof (as shown in figure 6), the verifier checks the commitment corresponding to each player except  $i^*$ , then simulates all the players except  $i^*$ . Since the verification is supposed to succeed, the verifier knows that  $[v_j] = [0]$  and  $[\mathbf{v}'] = [1]$ . So the value of the share of  $[v_j]$  and  $[\mathbf{v}']$  corresponding to  $i^*$  can be computed. This allows to do a final check to ensure that the challenges were fairly sampled by the prover. In the description of the

**Inputs:** The public key  $H = (I_{n-k}|T) \in \mathbb{F}_2^{n-k \times n}$ . The ciphertext  $(c_1, c_2)$  such that  $c_1 \in \mathbb{F}_2^{n-k}$ . The ciphertext randomness  $r = (r_1, \dots, r_n) \in \mathbb{F}_2^n$ , which is such that  $\text{wt}(r) = t$  and  $c_1 = Hr$ . We denote  $r_A = (r_1, \dots, r_{n-k})$ .

**Parameters:** A number  $\tau$  of iterations, a number  $N$  of players, a number  $s$  of evaluation points for the first challenge, finite field extensions  $\mathbb{F}_2 \subseteq \mathbb{F}_{poly} \subseteq \mathbb{F}_{points}$  and  $n$  distinct points  $\gamma_1, \dots, \gamma_n \in \mathbb{F}_{poly}$ . We denote  $F = \prod_{i=1}^n (X - \gamma_i)$ .

**Randomness:** Sample a root seed  $\text{seed}^{(e)} \leftarrow^{\$} \{0, 1\}^\lambda$  for  $1 \leq e \leq \tau$  and a salt  $\text{salt} \leftarrow^{\$} \{0, 1\}^{2\lambda}$ .

**Round 1.0:** Proof witness:

1. Compute  $E = \{i \in [1, n] \mid r_i \neq 0\}$  and  $Q = \prod_{i \in E} (X - \gamma_i)$ .
2. Compute  $S_r \in \mathbb{F}_{poly}[X]$  of degree  $\leq n - 1$  such that  $\forall i \in [1, n], S_r(\gamma_i) = r_i$ .
3. Compute  $P = S_r Q / F$ , and the Bézout coefficients  $U, V \in \mathbb{F}_{poly}[X]$  using the extended euclidian algorithm on input  $(S_r, F)$ . Set  $\mathbf{y} = (u_0, \dots, u_{t-1})$ .

**Round 1.1:** MPCitH inputs: for each  $1 \leq e \leq \tau$ :

1. Compute the players' seed  $(\text{seed}_i^{(e)})_{i \in [1, N]}$  using  $\text{TreePRG}(\text{salt}, \text{seed}^{(e)})$ .
2. For each player  $i \in [1, N]$ :
  - $([a_j^{(e)}]_i, [b_j^{(e)}]_i)_{j \in [1, s]}, [\mathbf{A}'^{(e)}]_i, [\mathbf{b}'^{(e)}]_i \leftarrow \text{PRG}(\text{salt}, \text{seed}_i^{(e)})$ .
  - if  $i \neq N$ :
    - $([c_j^{(e)}]_i)_{j \in [1, s]}, [\mathbf{c}'^{(e)}]_i, [r_A]_i^{(e)}, [P]_i^{(e)}, [Q]_i^{(e)}, [\mathbf{y}]_i^{(e)}, [V]_i^{(e)} \leftarrow \text{PRG}(\text{salt}, \text{seed}_i^{(e)})$ .
    - $\text{state}_i^{(e)} \leftarrow \text{seed}_i^{(e)}$ .
  - if  $i = N$ :
    - Compute  $[r_A]_N^{(e)}$  so that  $[r_A]^{(e)}$  is a share of  $r_A$ :  $[r_A]_N^{(e)} \leftarrow r_A - \sum_{l \neq N} [r_A]_l^{(e)}$ .
    - Proceed in the exact same way for  $[P]_N^{(e)}, [Q]_N^{(e)}, [\mathbf{y}]_N^{(e)}$ , and  $[V]_N^{(e)}$ .
    - $[c_j^{(e)}]_N \leftarrow a_j^{(e)} \cdot b_j^{(e)} - \sum_{l \neq N} [c_j^{(e)}]_l$  for  $j \in [1, s]$ .
    - $[\mathbf{c}'^{(e)}]_N \leftarrow \mathbf{A}'^{(e)} \mathbf{b}'^{(e)} - \sum_{l \neq N} [\mathbf{c}'^{(e)}]_l$ .
    - $\text{aux}^{(e)} \leftarrow ([r_A]_N^{(e)}, [P]_N^{(e)}, [Q]_N^{(e)}, [\mathbf{y}]_N^{(e)}, [V]_N^{(e)}, ([c_j^{(e)}]_N)_{j \in [1, s]}, [\mathbf{c}'^{(e)}]_N)$ .
    - $\text{state}_N^{(e)} \leftarrow \text{seed}_N^{(e)} \parallel \text{aux}^{(e)}$ .
  - Compute the commitment:  $\text{com}_i^{(e)} \leftarrow \mathcal{H}_0(\text{salt}, e, i, \text{state}_i^{(e)})$ .

**Round 2:** Set  $h_1 = ((\epsilon_j^{(e)}, \rho_j^{(e)})_{j \in [1, s]}, \epsilon'^{(e)})_{e \in [1, \tau]} \leftarrow \mathcal{H}_1(H, c_1, c_2, \text{salt}, (\text{com}_i^{(e)})_{e \in [1, \tau], i \in [1, N]})$ .

**Round 3:** MPCitH simulation: for each iteration  $1 \leq e \leq \tau$ :

1. Players locally set  $[r_B]^{(e)} \leftarrow y_1 - T_1[r_A]^{(e)}$ .
2. Players locally compute  $[S_r]^{(e)}$ , and  $[S_r \text{ quo } X^{n-t-i}]^{(e)}$  for  $0 \leq i \leq t-1$  thanks to  $[r]^{(e)} = [r_B | r_A]^{(e)}$ . They also locally compute  $[-FV \text{ quo } X^{n-t}]^{(e)}$  thanks to  $[V]^{(e)}$ .
3. for all  $j \in [1, s]$ :
  - Players locally evaluate shared polynomials on  $\rho_j^{(e)}$  to obtain the following shares:
 
$$[S_r(\rho_j^{(e)})], [Q(\rho_j^{(e)})], [FP(\rho_j^{(e)})], [\mathbf{z}_j^{(e)}] = [(-FV \text{ quo } X^{n-t})(\rho_j^{(e)})],$$
 and  $[\mathbf{X}_{i,j}^{(e)}] = [(\rho_j^{(e)})^i \cdot (S_r \text{ quo } X^{n-t-i})(\rho_j^{(e)})]$  for  $i \in [0, t-1]$ .
  - Simulate **ProdVerif** <sub>$d, D$</sub>  ( $d = 1, D = 1$ ) to verify  $([S_r(\rho_j^{(e)})], [Q(\rho_j^{(e)})], [FP(\rho_j^{(e)})])$ , using the triple  $([a_j^{(e)}], [b_j^{(e)}], [c_j^{(e)}])$  and  $\epsilon_j^{(e)}$ . Players obtain shares of  $[v_j^{(e)}]$  (we should have  $v_j^{(e)} = 0$ ), as well as shares of  $[\alpha_j^{(e)}]$ , and  $[\beta_j^{(e)}]$  (the masked values).
4. Simulate **ProdVerif** <sub>$d, D$</sub>  ( $d = t, D = s$ ) to verify  $([\mathbf{X}^{(e)}], [\mathbf{y}]^{(e)}, [\mathbf{z}^{(e)}])$ , using the triple  $([\mathbf{A}'^{(e)}], [\mathbf{b}'^{(e)}], [\mathbf{c}'^{(e)}])$  and  $\epsilon'^{(e)}$ . Players obtain shares of  $[\mathbf{v}'^{(e)}]$  (we should have  $\mathbf{v}'^{(e)} = \mathbf{1}$ ), as well as shares of  $[\alpha'^{(e)}]$ , and  $[\beta'^{(e)}]$  (the masked values).

**Round 4:**  
Set  $h_2 = (i^{*(e)})_{e \in [1, \tau]} \leftarrow \mathcal{H}_2(H, c_1, c_2, \text{salt}, h_1, (([\alpha_j, \beta_j, v_j])_{j \in [1, s]}^{(e)}, [\alpha'^{(e)}, \beta'^{(e)}, \mathbf{v}'^{(e)}])_{e \in [1, \tau]}).$

**Output:** Output the proof  $\pi$  as

$$\text{salt} \mid h_1 \mid h_2 \mid \left( (\text{state}_i^{(e)})_{i \neq i^{*(e)}} \mid [\text{com}_{i^{*(e)}}^{(e)}] \mid (([\alpha_j^{(e)}, \beta_j^{(e)}]_{j \in [1, s]}, \alpha'^{(e)}, \beta'^{(e)})_{i^{*(e)}})_{e \in [1, \tau]} \right).$$

**Fig. 5.** NIZK proof that a ciphertext  $(c_1, c_2)$  is well-formed: construction

proof system,  $a_j, b_j, \alpha_j, \beta_j, c_j$  and  $\epsilon_j$  are scalar elements of  $\mathbb{F}_{points}$ .  $\mathbf{A}'$  and  $\boldsymbol{\alpha}'$  are matrices of size  $t \times s$  in  $\mathbb{F}_{points}$ ,  $\mathbf{b}'$  and  $\boldsymbol{\beta}'$  are vectors of size  $t$  with coefficients in  $\mathbb{F}_{points}$ , and  $\mathbf{c}'$  and  $\boldsymbol{\epsilon}'$  are vectors of size  $s$  with coefficients in  $\mathbb{F}_{points}$ . All these elements are used to run the product verification.

**Inputs:** The public key  $H = (I_{n-k}|T) \in \mathbb{F}_2^{n-k \times n}$ . The ciphertext  $(c_1, c_2)$  such that  $c_1 \in \mathbb{F}_2^{n-k}$ . The proof  $\pi$ .

**Parameters:** A number  $\tau$  of iterations, a number  $N$  of players, a number  $s$  of evaluation points for the first challenge, finite field extensions  $\mathbb{F}_2 \subseteq \mathbb{F}_{poly} \subseteq \mathbb{F}_{points}$  and  $n$  distinct points  $\gamma_1, \dots, \gamma_n \in \mathbb{F}_{poly}$ . We denote  $F = \prod_{i=1}^n (X - \gamma_i)$ .

1. Parse the proof as  
 $\text{salt} \mid h_1 \mid h_2 \mid \left( (\text{state}_i^{(e)})_{i \neq i^*(e)} \mid \text{com}_{i^*(e)}^{(e)} \mid [(\alpha_j^{(e)}, \beta_j^{(e)})_{j \in [1, s]}, \boldsymbol{\alpha}'^{(e)}, \boldsymbol{\beta}'^{(e)}]_{i^*(e)} \right)_{e \in [1, \tau]}$ .
2. Compute the first challenges: parse  $h_1$  as  $((\epsilon_j^{(e)}, \rho_j^{(e)})_{j \in [1, s]}, \boldsymbol{\epsilon}'^{(e)})_{e \in [1, \tau]} \in (\mathbb{F}_{points}^{3s})^\tau$ .
3. Compute the second challenges: parse  $h_2$  as  $(i^*(e))_{e \in [1, \tau]} \in [1, N]^\tau$ .
4. Simulate the MPC in-the-head protocol: For each iteration  $e \in [1, \tau]$ :
  - For  $i \neq i^*(e)$ , compute  $\text{com}_i^{(e)} \leftarrow \mathcal{H}_0(\text{salt}, e, i, \text{state}_i^{(e)})$ .
  - For  $i \neq i^*(e)$ , using  $\text{state}_i^{(e)}$ , the elements  $(\epsilon_j^{(e)}, \rho_j^{(e)})$  and  $\boldsymbol{\epsilon}'^{(e)}$  of the first challenge, and  $[(\alpha_j^{(e)}, \beta_j^{(e)})_{j \in [1, s]}, \boldsymbol{\alpha}'^{(e)}, \boldsymbol{\beta}'^{(e)}]_{i^*(e)}$ , compute the view of player  $i$  as in the construction algorithm in order to compute  $([\alpha_j^{(e)}]_i, [\beta_j^{(e)}]_i, [v_j^{(e)}]_i)_{j \in [1, s]}$  and  $[\boldsymbol{\alpha}'^{(e)}]_i, [\boldsymbol{\beta}'^{(e)}]_i$ , and  $[\mathbf{v}'^{(e)}]_i$ .
  - Compute the final shares of player  $i^*(e)$  as  $[v_j^{(e)}]_{i^*(e)} \leftarrow - \sum_{i \neq i^*(e)} [v_j^{(e)}]_i$  for  $1 \leq j \leq s$  and  $[\mathbf{v}'^{(e)}]_{i^*(e)} \leftarrow \mathbf{1} - \sum_{i \neq i^*(e)} [\mathbf{v}'^{(e)}]_i$ .
5. Compute  $h'_1 \leftarrow \mathcal{H}_1(H, c_1, c_2, \text{salt}, (\text{com}_i^{(e)})_{e \in [1, \tau], i \in [1, N]})$ .
6. Compute  $h'_2 \leftarrow \mathcal{H}_2(H, c_1, c_2, \text{salt}, h_1, (([\alpha_j, \beta_j, v_j])_{j \in [1, s]}^{(e)}, [\boldsymbol{\alpha}'^{(e)}, \boldsymbol{\beta}'^{(e)}, \mathbf{v}'^{(e)}])_{e \in [1, \tau]}).$
7. Accept if and only if  $h_1 = h'_1$  and  $h_2 = h'_2$ .

**Fig. 6.** NIZK proof that a ciphertext  $(c_1, c_2)$  is well-formed: verification

**Properties of the proof system.** Theorems 3 and 4 state that the NIZK proof system is complete and zero-knowledge respectively. While the proof of Theorem 3 is immediate from the description of the protocols in Figures 5 and 6, we provide a complete proof of Theorem 4 in Appendix D. Finally, Theorem 5 states that the NIZK is one-time simulation straight-line extractable, which is the property that is needed for our conversion from OW-CPA to IND-CCA2 given in section 3. The proof of Theorem 5 is given in Appendix E for completeness.

**Theorem 3 (Completeness).** *The NIZK described by algorithms in figures 5 and 6 is perfectly complete; if an honest prover knows a vector  $r$  such that  $c_1 = Hr$ ,  $c_2 = Hr$  and  $\text{wt}(r) = t$  and uses it to build a proof  $\pi$ , then  $\pi$  will always convince an honest verifier (using the algorithm in figure 6).*

**Theorem 4 (Zero-knowledge).** *If the PRG used in the NIZK described by the algorithms in figures 5 and 6 is  $(t, \epsilon_{PRG})$ -secure and  $\mathcal{H}_0, \mathcal{H}_1$ , and  $\mathcal{H}_2$  are*

modeled as random oracles, then there exists a simulator  $\mathcal{S}$  such that: for any adversary running in time at most  $t$  making at most  $q_1$  queries to  $\mathcal{H}_1$  and  $q_2$  queries to  $\mathcal{H}_2$ , the advantage of distinguishing between the two games  $\mathbf{ZK}^s$  and  $\mathbf{ZK}^{\text{real}}$  (see definition 4) is at most  $\tau \cdot \epsilon_{\text{PRG}} + (q_1 + q_2)/2^{2\lambda}$ . Moreover, the running time of  $\mathcal{S}$  is at most twice the running time of the algorithm to build a proof (i.e. figure 5).

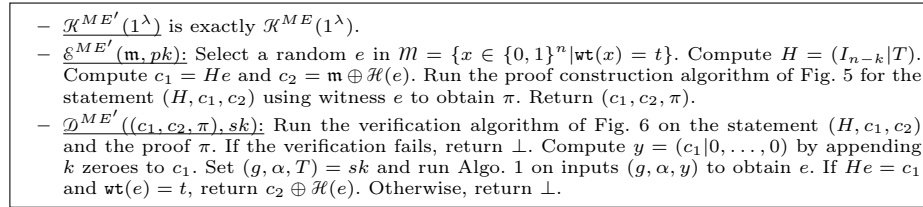
**Theorem 5 (One-time simulation extractability).** *Let  $\Delta = |\mathbb{F}_{\text{points}}|$ . There exists an extractor  $\mathcal{T}$  that has the following property: any adversary against the game  $\mathbf{Exp}^{\text{simextract}}$  (see definition 7) for the NIZK described by algorithms in figures 5 and 6, making at most  $q_0$ ,  $q_1$ , and  $q_2$  requests to the random oracles  $\mathcal{H}_0$ ,  $\mathcal{H}_1$ , and  $\mathcal{H}_2$  respectively, wins the game with probability at most:*

$$\frac{q_0}{2^{2\lambda}} + \frac{q_1}{\Delta^{2s\tau}} + \frac{q_2}{N^\tau} + q_2 \left( p + \frac{1}{N} \right)^\tau, \text{ where } p = \left( \frac{n+t}{\Delta} \right)^s. \quad (3)$$

Moreover, the extractor  $\mathcal{T}$  is running in time  $\mathcal{O}(q_0 + q_1 + q_2 + \tau N)$ .

### 4.3 An IND-CCA2 scheme based on the Niederreiter cryptosystem

Now that we have described our proof system corresponding to the correct language that is simulation straight-line extractable, we can instantiate corollary 1 using the Niederreiter cryptosystem (see figure 1) as the base OW-CPA scheme. The resulting conversion is given in figure 7, and corollary 1 applied to the NIZK presented above as well as the Niederreiter cryptosystem result in corollary 2.



**Fig. 7.** The Niederreiter cryptosystem using our generic conversion and the NIZK proof

**Corollary 2.** *Let  $\mathcal{H} : M \rightarrow \{0, 1\}^l$ ,  $\mathcal{H}_0 : \{0, 1\}^* \rightarrow \{0, 1\}^k$ ,  $\mathcal{H}_1 : \{0, 1\}^* \rightarrow (\mathbb{F}_{\text{points}})^{s \cdot \tau}$ , and  $\mathcal{H}_2 : \{0, 1\}^* \rightarrow \llbracket 1, N \rrbracket^\tau$  be hash functions modeled as a random oracles. The public key encryption scheme  $(\mathcal{K}^{ME'}, \mathcal{E}^{ME'}, \mathcal{D}^{ME'})$  described in figure 7 is  $(t, \epsilon)$ -IND-CCA2<sup>+</sup>, supposing that  $(\mathcal{K}^{ME}, \mathcal{E}^{ME}, \mathcal{D}^{ME})$  (see figure 1) is  $(t_{\text{ow}}, \epsilon_{\text{ow}})$ -OW-CPA, and that the PRG used in figures 5 and 6 is  $(t_{\text{PRG}}, \epsilon_{\text{PRG}})$ -*



secure, where:

$$\begin{aligned}\epsilon &= \epsilon_{ow} + 2 \left( \tau \cdot \epsilon_{PRG} + (q_1 + q_2)/2^{2\lambda} \right) + 2 \left( \frac{q_0}{2^\lambda} + \frac{q_1}{\Delta^{2s\tau}} + \frac{q_2}{N^\tau} + q_2 \left( p + \frac{1}{N} \right)^\tau \right) \\ t_{ow} &= t + 2\tau_{\mathcal{D}} + q_O(\tau_V + \tau_{\mathcal{T}}) + q_{\mathcal{H}}\tau_{\mathcal{E}}, \\ t_{PRG} &= t + \tau_{\mathcal{K}} + \tau_{\mathcal{E}} + q_O(\tau_V + 2\tau_{\mathcal{D}}) + q_{\mathcal{H}}\tau_{\mathcal{E}},\end{aligned}$$

where  $\tau_{\mathcal{K}}, \tau_{\mathcal{E}}, \tau_{\mathcal{D}}$  are the running times of  $\mathcal{K}^{ME}, \mathcal{E}^{ME}$ , and  $\mathcal{D}^{ME}$  respectively,  $\tau_{\mathcal{D}}$  (resp.  $\tau_V$ ) is the running time for algorithm 5 (resp. 6),  $\tau_{\mathcal{T}} = \mathcal{O}(q_0 + q_1 + q_2 + \tau N)$  is the running time of the simulator (see theorem 5) and  $q_O$  (resp.  $q_{\mathcal{H}}, q_0, q_1, q_2$ ) is an upper bound for the number of queries made by an adversary running in time at most  $t$  to the decryption oracle with randomness (resp. to the random oracles  $\mathcal{H}, \mathcal{H}_0, \mathcal{H}_1$ , and  $\mathcal{H}_2$ ).

## 5 Threshold decryption from multiparty Goppa decoding

In order to present a threshold scheme based on our non-threshold IND-CCA Niederreiter scheme (see Fig. 7), a multiparty protocol corresponding to the Patterson algorithm (see Algo. 1) is needed. In this section, we first present an MPC protocol for the decryption of the Niederreiter cryptosystem. Much like the protocol in [THO23], it uses results from [MF06] to compute some operations on shared polynomials. Our protocol improves over [THO23] by improving one of the two most costly step of the MPC protocol for decoding (i.e. the Patterson algorithm [Pat75]). This improvement allows for speedup for the steps before the rational reconstruction (this part of the secure computation only needs a quadratics number of operations instead of cubic in [THO23]) and it allows to shorten the shared secret key.

In [THO23], computing the square root of a polynomial (modulo an irreducible polynomial), corresponding to step 5 in algorithm 1, is done in a naïve way, i.e. the authors add a matrix representing this operation to the secret key. We introduce a technique, that we call "field switching", which consists in translating the operations mod  $[g]$ , where  $[g]$  is the shared irreducible polynomial corresponding to the secret Goppa code of the system, to some operations mod  $h$ , where  $h$  is an irreducible public polynomial of the same degree. This change leads to easier computations. For example, the operation  $[f] \mapsto [f^{1/2}]$  mod  $h$  is only linear in the shared input (since  $h$  is public), so computing this operation is free in terms of communication, and it only requires a linear amount of extra shared data.

We first present our field switching techniques for MPC, where we use techniques from computer algebra that are discussed mainly in [BDFD<sup>+</sup>19, Section 8] and [DFDS14]. Then, we present our complete MPC protocol for decoding, as well as our full threshold Niederreiter encryption.

## 5.1 Field switching techniques for MPC

Given a shared irreducible polynomial  $[g]$  of degree  $d$  with  $g \in \mathbb{F}_q[X]$ , our aim is to efficiently compute in a constant number of rounds some operations on shared inputs over the finite field  $\mathbb{F}_q[X]/g$ . The main difficulty lies in the fact that the description of the finite field, *i.e.* the irreducible polynomial  $g$ , is not public.

Indeed, if we assume that the field is constructed from a public irreducible polynomial  $h$  of degree  $d$ , all linear operations on shared inputs can be done without any communication while multiplications or inversions only require  $\mathcal{O}(d)$  secure multiplications. When the field is defined upon a shared private polynomial  $[g]$ , the inversion of a shared element  $[a]$  boils down to computing the gcd of  $[a]$  and  $[g]$ . Unfortunately, the best constant round protocol for the latter operation requires to invert the shared Sylvester matrix constructed from  $[a]$  and  $[g]$ , yielding a complexity of  $\mathcal{O}(d^3)$  secure multiplications [MF06]. The same situation arises for other operations such as the computation of the Frobenius map (or its inverse), *i.e.*  $[a^q]$  from a shared  $[a]$ , which does not require any communication when the field is defined with a public  $h$  while it could cost more than a quadratic number of shared products when a private shared  $[g]$  is used.

Our idea to speed up these shared operations is to use the isomorphism between the two extension fields  $\mathbb{K} = \mathbb{F}_q[X]/g$  and  $\mathbb{L} = \mathbb{F}_q[X]/h$  to transfer the operations from a private field setting with  $[g]$  to a public one with  $h$ . Let  $\phi : \mathbb{K} \rightarrow \mathbb{L}$  be the function that maps shared elements from one description to another. One can easily speed up operations on a shared element  $[a]$  by simply computing the same operation on  $[\phi(a)]$  and converting the result back to the original field  $\mathbb{K}$  (using  $\phi^{-1}$ ). Of course, one can use a shared matrix description of  $\phi$  and  $\phi^{-1}$  to compute the conversions through a matrix-vector product. However, this solution is not satisfying since the description is not compact enough: It requires  $\mathcal{O}(d^2)$  elements of  $\mathbb{F}_q$ .

We describe in the next sections two protocols that compute the map  $\phi$  and  $\phi^{-1}$  using only the knowledge of the shared polynomials  $\phi(x)$  and  $\phi^{-1}(z)$ , where  $x$  (resp.  $z$ ) denotes the class of  $X$  in  $\mathbb{K} = \mathbb{F}_q[X]/g$  (resp.  $\mathbb{L} = \mathbb{F}_q[X]/h$ ). These protocols are as efficient as the naive approach using matrix-vector product but they only require  $\mathcal{O}(d)$  elements of  $\mathbb{F}_q$  to describe  $\phi$  and  $\phi^{-1}$ .

**Computing  $\phi$  and  $\phi^{-1}$  through modular composition.** Since Shoup's work on polynomial factorization [Sho94, Sho95, Sho99], it is now well established that computing the evaluation of  $\phi$  reduces to the more general problem of *modular composition*, *i.e.*  $\phi(a) \bmod h$ . A general survey on algorithms evaluating embeddings of finite fields and their inverses can be found in [BDFD<sup>+</sup>19].

By linearity of  $\phi$  and of the sharing  $[\cdot]$ , we can rewrite  $[\phi(a)] = \phi(\sum_{i=0}^{d-1} [a_i X^i])$  as  $\sum_{i=0}^{d-1} [a_i] [\phi(X)]^i \bmod h$ . Writing  $[b] = [\phi(x)] \in \mathbb{L}$ , with  $x$  being the class of  $X$  in  $\mathbb{K} = \mathbb{F}_q[X]/g$ , we do have  $\phi(a) \bmod h = a(b) \bmod h$ .

If the shared polynomial  $[b]$  is already known, one can use an adaptation – to the MPC setting – of the famous modular composition algorithm of Brent and Kung [BK78]. Recalling that the algorithm is based on a Baby-Steps Giant-Steps

strategy, we assume without loss of generality that  $d = k^2$  for an integer  $k > 0$ . The first step is to compute the "small" powers of  $[b]$ , i.e.  $[b^2], \dots, [b^k] \bmod h$  (baby steps). Then, it computes the "big" powers of  $[b]$  by step of  $k$ , i.e.  $[b^{2k}], \dots, [b^{k(k-1)}] \bmod h$  (giant steps). The composition is finally computed as

$$[\phi(a) \bmod h] = \sum_{t=0}^{k^2-1} [a_t][b^t] \bmod h = \sum_{i=0}^{k-1} \left( [b^{ik}] \times \sum_{j=0}^{k-1} [a_{ik+j}][b^j] \right) \bmod h \quad (4)$$

The protocol **ModComp** described below computes  $[\phi(a)] \in \mathbb{L}$  for given  $[b] = [\phi(x)] \in \mathbb{L}$  and  $[a] \in \mathbb{K}$ . It is based on parallel invocations of constant round protocols for polynomial multiplication and division, each of which require  $\mathcal{O}(d)$  secure multiplications [MF06] (see section 2.4), and  $\mathcal{O}(d^2)$  shared products in  $\mathbb{F}_q$  for the most inner sum of Equation 4.

---

### Protocol 2: ModComp

---

**Parameters:**  $d \in \mathbb{N}$ ,  $h \in \mathbb{F}_q[X]$  of degree  $d$ ,  $k = \lceil \sqrt{d} \rceil$

**Input:**  $[a], [b] \in \mathbb{F}_q[X]$  of degree  $< d$

**Output:**  $[a(b) \bmod h]$

- 1 Parties compute  $[b^2], \dots, [b^k] \bmod h$  ▷ **FirstPowers** in  $\mathbb{F}_q[X]/h$
  - 2 Parties compute  $[b^{2k}], \dots, [b^{k^2-k}] \bmod h$  ▷ **FirstPowers** in  $\mathbb{F}_q[X]/h$
  - 3 In parallel,  $[\zeta_i] \leftarrow \sum_{j=0}^{k-1} [a_{ik+j}][b^j]$  for  $0 \leq i \leq k-1$
  - 4 In parallel,  $[\gamma_i] \leftarrow [\zeta_i][b^{ik}] \bmod h$  for  $0 \leq i \leq k-1$  ▷ **PolyMul** in  $\mathbb{F}_q[X]/h$
  - 5 Parties output  $[\alpha] \leftarrow \sum_{i=0}^{k-1} [\gamma_i]$
- 

**Theorem 6.** *Protocol **ModComp** is correct, private and requires a constant number of rounds and  $\mathcal{O}(d^2)$  secure multiplications in  $\mathbb{F}_q$ .*

Computing  $\phi^{-1}$  through our protocol **ModComp** is no more feasible since the computation is done modulo the private polynomial  $[g]$ . In particular, the underlying protocol **FirstPowers** requires to generate random invertible elements and their inverse in  $\mathbb{F}_q[X]/g$ , which could not be achieved efficiently because modular inverse with shared polynomial is too costly (that was our original motivation to transfer this operation through an isomorphism of finite fields).

Fortunately, we can circumvent this problem by decomposing several multiplications in  $\mathbb{F}_q[X]/g$  as several multiplications in  $\mathbb{F}_q[X]$  (letting the degree increase) followed by a single reduction modulo  $[g]$ . For the protocol **ModComp** above, this implies to see first  $[b]$  as a polynomial of  $\mathbb{F}_q[X]$  and to compute the polynomials  $[b], \dots, [b^k]$  in  $\mathbb{F}_q[X]$ , all being of degree at most  $kd = \mathcal{O}(d\sqrt{d})$ . One can use the constant round protocol from [GLOV24] for the latter operation with a total of  $\mathcal{O}(\tau d^{1.5+\frac{1}{\tau}}) = \mathcal{O}(d^2)$  secure multiplication for any integer  $\tau > 1$ . In order to map the result back in  $\mathbb{F}_q[X]/g$  it is sufficient to use the division protocol **PolyDiv** for the non-reduced polynomials  $([b^2], \dots, [b^k])$ .

Since the degree of each  $[b^k]$  is less than  $k^3$ , and that the division protocol requires a linear number of secure multiplication in the dividend's degree, the overall cost is  $k \times \mathcal{O}(k^3) = \mathcal{O}(d^2)$ . The same technique applies for computing  $([b^{2k}, \dots, b^{(k(k-1))}])$  since  $[b^k] \bmod [g]$  is known from the previous computation. For the last polynomial multiplication of the protocol, we can also apply the same lazy reduction modulo  $[g]$ .

Applying these slight modifications to **ModComp** when the parameter  $h$  is no more public and shared among parties, we obtain a protocol that is still constant round (i.e.  $\mathcal{O}(\tau)$  for any fixed integer  $\tau > 1$ ) and that requires  $\mathcal{O}(d^2)$  secure multiplications in  $\mathbb{F}_q$ .

## 5.2 Multiparty Goppa decoding

With our field-switching techniques, we can now present our full MPC protocol for decoding a Goppa code. The protocol takes as input the shared parameters  $([g], [\alpha])$  of the code, as well as a public noisy codeword  $y = c + e$  for which the error is of Hamming weight exactly  $t$ . At the end of the protocol,  $e$  is revealed to all the participants. Using this protocol as the main building block, we present our IND-CCA2 threshold cryptosystem based on the Niederreiter cryptosystem.

Algorithm 1 has three parts that are challenging to do efficiently in MPC: steps 1 through 4 for computing the syndrome  $s$ , step 5 for computing the element  $u$  used for the rational reconstruction, and the rational reconstruction itself at step 6. To compute these steps as efficiently as possible and with as few precomputations as possible, our protocol for decoding requires using protocols from section 5.1. More precisely, the protocol first computes  $[\mu] = [\phi(u)]$  to compute  $[u]$ . The operations are therefore done in  $\mathbb{F}_{2^m}[X]/h$  (remember that  $h$  is public) rather than in  $\mathbb{F}_{2^m}[X]/g$ . This requires that the participants share extra elements beyond  $([g], [\alpha])$ . Namely, they need to agree on a public irreducible polynomial  $h$  of the same size as  $[g]$ , as well as the shared polynomials  $[\phi(x)]$  and  $[\phi^{-1}(z)]$ , where  $\phi$  is a (secret) isomorphism from  $\mathbb{F}_{2^m}[X]/g$  to  $\mathbb{F}_{2^m}[X]/h$  and  $x$  (resp.  $z$ ) is the class of  $X$  in  $F_q[X]/g$  (resp  $F_q[X]/h$ ). While  $h$  can be considered as a parameter of the cryptosystem (the same value for  $h$  can be used for all possible keys), both  $[\phi(x)]$  and  $[\phi^{-1}(z)]$  need to be part of the secret key to be able to decrypt, and are taken as extra inputs for the protocol.

**Theorem 7.** *Protocol **DecodingMPC** is correct, private and requires a constant number of rounds and  $\mathcal{O}(t^3 + nt)$  secure multiplications.*

## 5.3 Putting everything together: our code-based threshold PKE

We are now ready to present the threshold cryptosystem. In essence, it is very similar to the PKE presented in figure 7, except decoding which is done in MPC using Protocol **DecodingMPC**. The partial key of each participant corresponds to their own share of  $[g]$  and  $[\alpha]$ , which are the shared parameters of the secret Goppa code, as well as  $[b], [c]$  as defined in **DecodingMPC**. Remember that  $[b] \in \mathbb{F}_q[X]_{<t}$  and  $[c] \in \mathbb{F}_q[X]_{<t}$  corresponds to extra information that allows the

---

**Protocol 3: DecodingMPC**


---

**Parameters:**  $n, m, t \in \mathbb{N}$  with  $t$  even,  $h \in \mathbb{F}_{2^m}[X]$  irreducible of degree  $t$ .  
**Input:**  $[g] \in \mathbb{F}_{2^m}[X]$  irreducible polynomial of degree  $t$ ,  
 $[\alpha] = ([\alpha_1], \dots, [\alpha_n]) \in \mathbb{F}_{2^m}^n$ ,  $[b] = [\phi(x)]$ ,  $[c] = [\phi^{-1}(z)]$ , and  $y \in \mathbb{F}_2^n$   
such that there exists  $e$  such that  $\text{wt}(e) = t$  and  $y - e \in \mathcal{C}(g, \alpha)$ .  
**Output:**  $e \in \mathbb{F}_2^n$  as a publicly known vector

- 1 **for**  $i$  **from** 1 **to**  $n$  **in parallel do**
- 2   | Parties compute  $[\gamma_i] \leftarrow [\phi((x - \alpha_i)^{-1})] = [(b - \alpha_i)^{-1}] \pmod{h}$    ▷ **FieldInv**
- 3 **end**
- 4  $[\sigma] \leftarrow [\phi(\sum_{i=1}^n y_i(x - \alpha_i)^{-1})] = \sum_{i=1}^n y_i[\gamma_i] \pmod{h}$    ▷ no communication
- 5 Parties compute  $[\sigma^{-1}] \pmod{h}$    ▷ **FieldInv**
- 6  $[\mu] \leftarrow [\sigma^{-1} + b]^{1/2} \pmod{h} = [\phi((s^{-1} + x)^{1/2})]$    ▷ no communication
- 7  $[u] \leftarrow [\phi^{-1}(\mu)] = [\mu(c) \pmod{g}]$    ▷ **ModComp** with shared mod.
- 8 Parties compute  $[r]$  and  $[v]$  such that  $uv = r \pmod{g}$  s.t.  $\deg(r) = t/2$ , and  
 $\deg(v) = t/2 - 1$    ▷ **RatRecon**
- 9  $[\epsilon] \leftarrow [r]^2 + X[v]^2$    ▷ no communication
- 10 Parties compute  $[\epsilon(\alpha_i)]$  for  $1 \leq i \leq n$    ▷ **FastEval**
- 11 Parties reveal  $e_i = (\epsilon(\alpha_i) \stackrel{?}{=} 0)$  for  $1 \leq i \leq n$    ▷ **EqTesting**
- 12 Parties output  $e = (e_1, \dots, e_n)$

---

MPC for decoding to be efficient, as explained in sub-section 5.1. The encryption is identical as in figure 7, i.e. the sender samples a random error vector whose hash is used as one-time pad, encrypts the error vector as a syndrome and builds a proof for the syndrome according to algorithm 5. For decryption, the participants act as follows: Upon receiving a ciphertext, all the participants check that the proof is correct. If all participants agree that the proof is correct, then Protocol 3 is run (using the shared secret key as input) and all parties obtain the error vector  $e$ . Using  $e$ , each participant can locally compute the plaintext. We detail the threshold system in figure 8. As in [THO23], we do not present a dedicated MPC protocol for the key generation. The question of designing a dedicated MPC protocol to generate a random irreducible polynomial of a given degree is an interesting question that could be the topic of future works. The cost of running a generic MPC protocol for the key generation is mitigated by the fact that it is only done once to generate the keys and then not used again for encryption or decryption.

As long as Protocol **DecodingMPC** does not fail, no information about the secret key is revealed. Moreover, the NIZK provided with each ciphertext guarantees that the input for the protocol is always correct (except with negligible probability). This ensures that every execution of the MPC protocol does not leak any information about the secret key, so that the security guarantees still hold for our threshold version of the PKE. Theorem 8 follows.

We denote  $(\mathcal{K}^{ME_T}, \mathcal{E}^{ME_T}, \mathcal{D}^{ME_T})$  the threshold public-key encryption scheme obtained by replacing the decryption algorithm in  $(\mathcal{K}^{ME'}, \mathcal{E}^{ME'}, \mathcal{D}^{ME'})$  by an interactive protocol. (see Figure 8). Basically, on input a ciphertext  $(c_1, c_2, \pi)$ ,

- $\mathcal{K}^{ME_T}(1^\lambda)$ : Each parties deduce locally  $(n, m, t)$ . Parties either run a generic MPC or have a trusted party run an algorithm to compute  $([g], [\alpha])$  such that  $g \in \mathbb{F}_{2^m}[X]$  is irreducible, monic, and of degree  $t$ , as well as the extra informations  $[b], [c]$  (needed in protocol **DecodingMPC**). Each party obtain the public key  $pk = H$  and their own share of the secret key  $[sk] = ([g], [\alpha], [b], [c])$ .
- $\mathcal{E}^{ME_T}(\mathbf{m}, pk)$  is exactly  $\mathcal{E}^{ME'}(\mathbf{m}, pk)$ .
- $\mathcal{D}^{ME_T}((c_1, c_2, \pi), H, ([g], [\alpha], [b], [c]))$ : Each party runs the verification algorithm of figure 6 on the statement  $(H, c_1, c_2)$  and the proof  $\pi$ . If at least one party has not successfully verified the proof, return  $\perp$ . Otherwise, each party locally compute  $y = (c_1|0, \dots, 0)$  by appending  $c_1$  by  $k$  zeroes. Then, parties run Protocol **DecodingMPC** on inputs  $([g], [\alpha], [b], [c], y)$  to obtain  $e$  (publicly). If  $He = c_1$  and  $\text{wt}(e) = t$ , return  $c_2 \oplus \mathcal{H}(e)$ . Otherwise, return  $\perp$ .

**Fig. 8.** Our threshold PKE based on the Niederreiter cryptosystem

each decryptor checks the validity of the proof  $\pi$  before running an instantiation of the multiparty Protocol **DecodingMPC** to obtain the syndrome which is output publicly and used to recover the plaintext.

**Theorem 8.** *Let  $\Pi$  be an instantiation of Protocol **DecodingMPC** that is  $(T, \epsilon_{MPC})$ -private (for some  $T < N$ ) in the semi-honest model (resp. malicious model). Assume that  $(\mathcal{K}^{ME'}, \mathcal{E}^{ME'}, \mathcal{D}^{ME'})$  is  $(\tau_{ME'}, \epsilon_{ME'})$ -IND-CCA2+ secure (See Corollary 2) for some  $0 < \epsilon_{ME'} < 1$ , then  $(\mathcal{K}^{ME_T}, \mathcal{E}^{ME_T}, \mathcal{D}^{ME_T})$  is  $(T, \epsilon)$ -IND-CCA2 secure in the semi-honest model (resp. malicious model) against any IND-CCA2 adversary making at most  $q_D$  (interactive) decryption query where  $\epsilon \leq \epsilon_{ME'} + q_D \cdot \epsilon_{MPC}$  and  $\tau_{ME_T} \leq \tau_{ME'} - q_D \cdot \tau_S$ , where  $\tau_S$  denotes the running time of the simulator  $\mathcal{S}$  for  $\Pi$ .*

*Proof (Sketch).* The reduction to the IND-CCA2+ security of  $(\mathcal{K}^{ME'}, \mathcal{E}^{ME'}, \mathcal{D}^{ME'})$  is straightforward. It simply uses the same public key and forwards the challenge query. For the simulation of the decryption oracle, by assumption on  $\Pi$ , there exists a simulator  $\mathcal{S}$  that can be used to simulate the outcome of a real execution of the decoding protocol **DecodingMPC** with an adversary  $\mathcal{A}$  (with a distribution  $\epsilon_{MPC}$ -close to a real execution) given only the output of the computation (namely  $e$ ). For each decryption query, the reduction for the  $(T, \epsilon)$ -IND-CCA2 security of  $(\mathcal{K}^{ME_T}, \mathcal{E}^{ME_T}, \mathcal{D}^{ME_T})$  simply calls the IND-CCA2+ oracle for  $(\mathcal{K}^{ME_T}, \mathcal{E}^{ME_T}, \mathcal{D}^{ME_T})$  to obtain the value  $e$  and then runs the simulator  $\mathcal{S}$  in the semi-honest model (resp. malicious model) to produce an execution of the interactive decryption protocol that is  $\epsilon_{MPC}$ -close to a real one.

## 6 Efficiency Considerations

In this section, we provide an evaluation of the efficiency of our new scheme, as well as comparisons with the only other code-based schemes with equivalent security from [THO23]. Even though we showed that their third scheme is flawed, we still present it in section 6 to highlight the fact that we obtain a scheme that has the same communication complexity, shorter secret key size, quite larger

ciphertexts (also independent from the number of parties  $N$  or threshold  $T$ ), but which achieves IND-CCA2 security.

**Choice of parameters.** In the following, for each of the Classic McEliece parameter set [ABC<sup>+</sup>22], we explain our choice for concrete values for all the different parameters in our scheme and provide in Table 1 the resulting sizes for the secret key, as well as ciphertext expansion, for all the Classic McEliece parameter sets.

The parameter sets give us values for  $n$  (the length of a codeword),  $m$  (the degree of the extension field of the Goppa code), and  $t$  (the Hamming weight of the error). Since  $k = n - mt$ , we also have values for  $k$ . The size of (partial) the secret keys is  $m(n + 3t)$  bits, so computing its concrete value is straightforward. The ciphertext size is given by  $|c_{\text{CM}}| + |\pi_{\mathcal{L}}|$  (where  $c_{\text{CM}}$  is a classic McEliece ciphertext and  $\pi_{\mathcal{L}}$  is our NIZK proof). Recall that the size of  $c_{\text{CM}}$  is  $mt$  bits. Computing the size of  $\pi_{\mathcal{L}}$  is trickier, as it depends on its own set of parameters (in addition to the parameters corresponding to the Niederreiter scheme). It depends on  $N$ ,  $\tau$ ,  $s$ ,  $|\mathbb{F}_{\text{poly}}|$ , and  $|\mathbb{F}_{\text{points}}|$  (cf. 4). Its complete length is given by the following formula, where  $\delta = |\mathbb{F}_{\text{poly}}|$  and  $\Delta = |\mathbb{F}_{\text{points}}|$ :

$$|\pi_{\mathcal{L}}| = 4\lambda + \tau(2\lambda + \log_2(N) + n - k + (4t - 1)\log_2(\delta) + ((6 + t)s + t)\log_2(\Delta)). \quad (5)$$

To give concrete values for  $N$ ,  $\tau$ ,  $s$ ,  $\delta$ , and  $\Delta$ , we follow the remarks given in [FJR22], as since our NIZK proof system follows the same security constraints as theirs. We also consider improvements from [AGH<sup>+</sup>23], in which the authors present a compression technique for the proof system of [FJR22] called Hypercubing. In essence, their technique consists of bundling the inputs of the MPCitH players in different ways, such that one MPCitH protocol over many players can be replaced by multiple MPCitH protocols over 2 players. For the same number of players  $N$  for the MPCitH protocols, this allows much faster computations for building and verifying the proofs with the same soundness error. In our case, applying the techniques of [AGH<sup>+</sup>23] is straightforward. It allows us to chose the number  $N$  of players to be larger than 256, which was the largest value of  $N$  considered in [FJR22]. Looking at [AGH<sup>+</sup>23, Table 7], it seems like the computational costs for  $N = 256$  and  $\tau = 16$  without Hypercubing are roughly the same as the computational costs for  $N = 2^{12}$  and  $\tau = 11$  with Hypercubing. We therefore chose these values as well, as it allows the soundness error  $(p + 1/N)^\tau$  to be less than  $2^{-\lambda}$  for  $\lambda = 128$ , so that our proof achieves 128-bits security. Next, we chose  $|\mathbb{F}_{\text{poly}}|$  to be the smallest power of two bigger than  $n$ , as it is the only constraint for this value. Finally, due to an attack described in [KZ20] on 5-rounds NIZK proof systems, the author of [FJR22] take  $s$  and  $\Delta$  such that the probability  $p = ((n + t)/\Delta)^s$  is negligible compared to  $1/N$ , and we follow their choice as well.

For the different McEliece parameters, we give in Table 1 our choices of values for the different parameters as well as the resulting sizes of our scheme. In the table, the total ciphertext size would be the sum of both the length of

the Niederreiter ciphertext and the size of our proof  $\pi_{\mathcal{L}}$ . The total length comes almost entirely from the NIZK proof  $\pi_{\mathcal{L}}$ . A noticeable part of the size of the proof is due to the term in  $ts \log(\Delta)$  in equation 5. This term is due to the product verification **ProdVerif** <sub>$d,D$</sub>  used to verify equation 2 in the MPCitH protocol.

We note that many parameters can be tuned according to the target trade-off between computational cost and communication complexity, and that other improvements exists for this kind of NIZK proofs (like [Bot22]’s compression techniques). Having a precise evaluation of the efficiency requires an implementation of the protocol, which is an interesting work on his own, and could be the topic of future works.

| Parameter set   | $N$      | $\tau$ | $s$ | $\delta$ | $\Delta$ | $p$       | security   | $ sk $  | $ \pi_{\mathcal{L}} $ | $ c_{CM} $ |
|-----------------|----------|--------|-----|----------|----------|-----------|------------|---------|-----------------------|------------|
| mceliece348864  | $2^{12}$ | 11     | 5   | $2^{12}$ | $2^{25}$ | $2^{-66}$ | $2^{-132}$ | 5.5 KB  | 21.6 KB               | 96 B       |
| mceliece460896  | $2^{12}$ | 11     | 5   | $2^{13}$ | $2^{26}$ | $2^{-69}$ | $2^{-132}$ | 8.0 KB  | 32.4 KB               | 156 B      |
| mceliece6688128 | $2^{12}$ | 11     | 5   | $2^{13}$ | $2^{26}$ | $2^{-66}$ | $2^{-132}$ | 11.5 KB | 42.1 KB               | 208 B      |
| mceliece6960119 | $2^{12}$ | 11     | 5   | $2^{13}$ | $2^{26}$ | $2^{-66}$ | $2^{-132}$ | 11.9 KB | 39.8 KB               | 227 B      |
| mceliece8192128 | $2^{12}$ | 11     | 5   | $2^{13}$ | $2^{26}$ | $2^{-64}$ | $2^{-132}$ | 13.9 KB | 44.2 KB               | 396 B      |

**Table 1.** Parameter choices and sizes according to the Classic McEliece parameter sets

**Comparison with other code-based threshold PKEs.** In the following, we compare the different sizes corresponding to our scheme with the schemes  $\Pi_t^I$ ,  $\Pi_t^{\Pi}$  and  $\Pi_t^{\text{III}}$  presented in [THO23]. It is worth recalling that the scheme  $\Pi_t^{\text{III}}$  doesn’t achieve IND-CCA2 security as explained in the introduction. A brief description of these schemes is given in Appendix F. We give a comparison depending on the Classic McEliece parameters in Table 2 (where  $|sk|$  denotes the size of a Classic McEliece secret key,  $|c_{CM}|$  the size of a ciphertext,  $|\sigma|$  and  $|vk|$  the size of a one-time signature and its verification key, and  $\ell_h$  and  $\ell_g$  output length of hash functions). We provide concrete values in Table 3.

|                       | secret key size          | ciphertext expansion                                | Communication complexity                 |
|-----------------------|--------------------------|-----------------------------------------------------|------------------------------------------|
| $\Pi_t^I$             | $ sk $                   | $\mathbb{N}( c_{CM}  + 4\lambda) +  \sigma  +  vk $ | —                                        |
| $\Pi_t^{\Pi}$         | $\binom{n-1}{t-2} sk $   | $\binom{n}{t-1}( c_{CM}  + \ell_h) + \ell_g$        | $\mathcal{O}(\binom{n}{t-1}n)$           |
| $\Pi_t^{\text{III}*}$ | $(nt + t^2)m = t sk $    | $ c_{CM}  + \ell_h + \ell_g$                        | $\mathcal{O}(t^3 + nt) + \mathcal{O}(n)$ |
| Ours                  | $(n + 3t)m =  sk  + 2tm$ | $ c_{CM}  +  \pi_{\mathcal{L}} $                    | $\mathcal{O}(t^3 + nt)$                  |

**Table 2.** Asymptotic comparison with schemes from [THO23].

\* $\Pi_t^{\text{III}}$  does not achieve IND-CCA2 security.



To give concrete values for their schemes, the authors of [THO23] chose the parameter set mceliece348864 for the values of  $n$ ,  $m$  and  $t$ . For the signature  $\sigma$  and its verification key  $vk$ , they use the signature from [BGKM23], which gives them  $|\sigma| = 15$  KB and  $|vk| = 165$  B. They also use various hash functions which gives them  $\ell_g = 32$  B and  $\ell_h = 64$  B. The NIST call for multi-party threshold schemes defines 6 regimes of numbers of parties: (Two ( $N = 2$ ), Three ( $N = 3$ ), Small ( $4 \leq N \leq 8$ ), Medium ( $9 \leq N \leq 64$ ), Large ( $65 \leq N \leq 1024$ ), and Enormous ( $N \geq 1025$ )).

In Table 3, we compare our scheme with  $\Pi_t^I$  and  $\Pi_t^{II}$ , and  $\Pi_t^{III}$  (instantiated with the parameter set mceliece348864) for various values of  $(N, T)$ . We show that the global cost to achieve IND-CCA2 security and distributed decryption for the McEliece cryptosystem is quite low: In the "Three" and "Small" regimes, the difference in ciphertext size is only tens of kilobytes with the other schemes. For higher values of  $N$ , our scheme is the only one achieving IND-CCA2 security whose ciphertext size is independent of  $(N, T)$ . The scheme  $\Pi_t^{II}$  rapidly becomes totally unpractical, and from the "Large" regime, our scheme becomes competitive with  $\Pi_t^I$ , as our schemes produces smaller ciphertext for  $N = 64$  and above. For secret key size, our scheme has a secret key size that is only 0.2 KB larger than the smallest size possible for  $\Pi_t^I$  and  $\Pi_t^{II}$ . Moreover, even though  $\Pi_t^{III}$  doesn't achieve chosen-ciphertext security, we still achieve a way better secret key size.

| (N, T)      | secret key size |              |               |        | ciphertext expansion |              |               |         |
|-------------|-----------------|--------------|---------------|--------|----------------------|--------------|---------------|---------|
|             | $\Pi_t^I$       | $\Pi_t^{II}$ | $\Pi_t^{III}$ | Ours   | $\Pi_t^I$            | $\Pi_t^{II}$ | $\Pi_t^{III}$ | Ours    |
| (3, 2)      | 5.3 KB          | 5.3 KB       | 346 KB*       | 5.5 KB | 15.6 KB              | 512 B        | 192 B*        | 21.8 KB |
| (8, 4)      |                 | 112 KB       |               |        | 16.4 KB              | 9.0 KB       |               |         |
| (64, 32)    |                 | $2^{71}$ B   |               |        | 25.4 KB              | $2^{67}$ B   |               |         |
| (128, 64)   |                 | $2^{135}$ B  |               |        | 35.6 KB              | $2^{131}$ B  |               |         |
| (512, 256)  |                 | $2^{518}$ B  |               |        | 97.0 KB              | $2^{514}$ B  |               |         |
| (1024, 512) |                 | $2^{1030}$ B |               |        | 179 KB               | $2^{1025}$ B |               |         |

**Table 3.** Efficiency comparisons with schemes from [THO23].

\*  $\Pi_t^{III}$  does not achieve IND-CCA2 security.

## References

- ABC<sup>+</sup>22. Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
- AGH<sup>+</sup>23. Carlos Aguilar-Melchor, Nicolas Gama, James Howe, Andreas Hülsing, David Joseph, and Dongze Yue. The return of the SDitH. In Carmit

- Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 564–596. Springer, Cham, April 2023.
- BBGK24. Slim Bettaleb, Loïc Bidoux, Philippe Gaborit, and Mukul Kulkarni. Modelings for generic PoK and applications: Shorter SD and PKP based signatures. *Cryptology ePrint Archive*, Paper 2024/1668, 2024.
- BCD<sup>+</sup>24. Lennart Braun, Guilhem Castagnos, Ivan Damgård, Fabien Laguillaumie, Kelsey Melissaris, Claudio Orlandi, and Ida Tucker. An improved threshold homomorphic cryptosystem based on class groups. In *Security and Cryptography for Networks: 14th International Conference, SCN 2024, Amalfi, Italy, September 11–13, 2024, Proceedings, Part II*, page 24–46, Berlin, Heidelberg, 2024. Springer-Verlag.
- BD10. Rikke Bendlin and Ivan Damgård. Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 201–218. Springer, Berlin, Heidelberg, February 2010.
- BDFD<sup>+</sup>19. Ludovic Brieulle, Luca De Feo, Javad Doliskani, Jean Pierre Flori, and Éric Schost. Computing isomorphisms and embeddings of finite fields. *ACM Commun. Comput. Algebra*, 52(4):117–119, may 2019.
- Bea92. Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 420–432. Springer, Berlin, Heidelberg, August 1992.
- Ber22. Daniel J. Bernstein. Understanding binary-goppa decoding. *Cryptology ePrint Archive*, Report 2022/473, 2022.
- BGKM23. Loïc Bidoux, Philippe Gaborit, Mukul Kulkarni, and Víctor Mateu. Code-based signatures from new proofs of knowledge for the syndrome decoding problem. *DCC*, 91(2):497–544, 2023.
- BHH<sup>+</sup>19. Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 61–90. Springer, Cham, December 2019.
- BIB89. Judit Bar-Ilan and Donald Beaver. Non-cryptographic fault-tolerant computing in constant number of rounds of interaction. In Piotr Rudnicki, editor, *8th ACM PODC*, pages 201–209. ACM, August 1989.
- BK78. R. P. Brent and H. T. Kung. Fast algorithms for manipulating formal power series. *J. ACM*, 25(4):581–595, oct 1978.
- BN20. Carsten Baum and Ariel Nof. Concretely-efficient zero-knowledge arguments for arithmetic circuits and their application to lattice-based cryptography. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part I*, volume 12110 of *LNCS*, pages 495–526. Springer, Cham, May 2020.
- Bot22. Cristian-Alexandru Botocan. One network to rule them all. an autoencoder approach to encode datasets. *Cryptology ePrint Archive*, Report 2022/890, 2022.
- BPR24. Dan Boneh, Aditi Partap, and Lior Rotem. Accountability for misbehavior in threshold decryption via threshold traitor tracing. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 317–351. Springer, Cham, August 2024.
- BS23. Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and

- Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 371–404. Springer, Singapore, December 2023.
- CCMS21. Kelong Cong, Daniele Cozzo, Varun Maram, and Nigel P. Smart. Gladius: LWR based efficient hybrid public key encryption with distributed decryption. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 125–155. Springer, Cham, December 2021.
- CD01. Ronald Cramer and Ivan Damgård. Secure distributed linear algebra in a constant number of rounds. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 119–136. Springer, Berlin, Heidelberg, August 2001.
- CLT22. Guilhem Castagnos, Fabien Laguillaumie, and Ida Tucker. Threshold linearly homomorphic encryption on  $\mathbf{Z}/2^k\mathbf{Z}$ . In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 99–129. Springer, Cham, December 2022.
- DDFY94. Alfredo De Santis, Yvo Desmedt, Yair Frankel, and Moti Yung. How to share a function securely. In *26th ACM STOC*, pages 522–533. ACM Press, May 1994.
- Des93. Yvo Desmedt. Threshold cryptosystems (invited talk). In Jennifer Seberry and Yuliang Zheng, editors, *AUSCRYPT’92*, volume 718 of *LNCS*, pages 3–14. Springer, Berlin, Heidelberg, December 1993.
- DF90. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990.
- DFDS14. Luca De Feo, Javad Doliskani, and Éric Schost. Fast arithmetic for the algebraic closure of finite fields. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, ISSAC ’14, page 122–129, New York, NY, USA, 2014. Association for Computing Machinery.
- DFK<sup>+</sup>06. Ivan Damgård, Matthias Fitzi, Eike Kiltz, Jesper Buus Nielsen, and Tomas Toft. Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In Shai Halevi and Tal Rabin, editors, *TCC 2006*, volume 3876 of *LNCS*, pages 285–304. Springer, Berlin, Heidelberg, March 2006.
- DJ01. Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 119–136. Springer, Berlin, Heidelberg, February 2001.
- DK05. Yevgeniy Dodis and Jonathan Katz. Chosen-ciphertext security of multiple encryption. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 188–209. Springer, Berlin, Heidelberg, February 2005.
- DLN<sup>+</sup>21. Julien Devevey, Benoît Libert, Khoa Nguyen, Thomas Peters, and Moti Yung. Non-interactive CCA2-secure threshold cryptosystems: Achieving adaptive security in the standard model without pairings. In Juan Garay, editor, *PKC 2021, Part I*, volume 12710 of *LNCS*, pages 659–690. Springer, Cham, May 2021.
- DLO<sup>+</sup>18. Ivan Damgård, Ji Luo, Sabine Oechsner, Peter Scholl, and Mark Simkin. Compact zero-knowledge proofs of small Hamming weight. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 530–560. Springer, Cham, March 2018.
- Fis05. Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *CRYPTO 2005*, vol-

- ume 3621 of *LNCS*, pages 152–168. Springer, Berlin, Heidelberg, August 2005.
- FJR22. Thibault Feneuil, Antoine Joux, and Matthieu Rivain. Syndrome decoding in the head: Shorter signatures from zero-knowledge proofs. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 541–572. Springer, Cham, August 2022.
- FJR23. Thibault Feneuil, Antoine Joux, and Matthieu Rivain. Shared permutation for syndrome decoding: new zero-knowledge protocol and code-based signature. *Des. Codes Cryptogr.*, 91(2):563–608, 2023.
- FP01. Pierre-Alain Fouque and David Pointcheval. Threshold cryptosystems secure against chosen-ciphertext attacks. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 351–368. Springer, Berlin, Heidelberg, December 2001.
- FPS01. Pierre-Alain Fouque, Guillaume Poupard, and Jacques Stern. Sharing decryption in the context of voting or lotteries. In Yair Frankel, editor, *FC 2000*, volume 1962 of *LNCS*, pages 90–104. Springer, Berlin, Heidelberg, February 2001.
- GG03. Joachim Gathen and Jürgen Gerhard. *Modern computer algebra (2. ed.)*. Cambridge University Press, 01 2003.
- GJKR07. Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, January 2007.
- GLOV24. Pascal Giorgi, Fabien Laguillaumie, Lucas Ottow, and Damien Vergnaud. Fast Secure Computations on Shared Polynomials and Applications to Private Set Operations. In Divesh Aggarwal, editor, *5th Conference on Information-Theoretic Cryptography (ITC 2024)*, volume 304 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 11:1–11:24, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- GRJK00. Rosario Gennaro, Tal Rabin, Stanislaw Jarecki, and Hugo Krawczyk. Robust and efficient sharing of RSA functions. *Journal of Cryptology*, 13(2):273–300, March 2000.
- HMR<sup>+</sup>19. Carmit Hazay, Gert Læssøe Mikkelsen, Tal Rabin, Tomas Toft, and Angelo Agatino Nicolosi. Efficient RSA key generation and threshold paillier in the two-party setting. *Journal of Cryptology*, 32(2):265–323, April 2019.
- KKW18. Jonathan Katz, Vladimir Kolesnikov, and Xiao Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 525–537. ACM Press, October 2018.
- KZ20. Daniel Kales and Greg Zaverucha. An attack on some signature schemes constructed from five-pass identification schemes. In Stephan Krenn, Haya Shulman, and Serge Vaudenay, editors, *CANS 20*, volume 12579 of *LNCS*, pages 3–22. Springer, Cham, December 2020.
- KZ22. Daniel Kales and Greg Zaverucha. Efficient lifting for shorter zero-knowledge proofs and post-quantum signatures. Cryptology ePrint Archive, Report 2022/588, 2022.
- Lin03a. Yehuda Lindell. General composition and universal composability in secure multi-party computation. In *44th FOCS*, pages 394–403. IEEE Computer Society Press, October 2003.

- Lin03b. Yehuda Lindell. A simpler construction of cca2-secure public-key encryption under general assumptions. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 241–254. Springer, Berlin, Heidelberg, May 2003.
- LN17. Yehuda Lindell and Ariel Nof. A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 259–276. ACM Press, October / November 2017.
- LP01. Anna Lysyanskaya and Chris Peikert. Adaptive security in the threshold setting: From cryptosystems to signature schemes. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 331–350. Springer, Berlin, Heidelberg, December 2001.
- LY20. Benoît Libert and Moti Yung. Adaptively secure non-interactive CCA-secure threshold cryptosystems: Generic framework and constructions. *Journal of Cryptology*, 33(4):1405–1441, October 2020.
- McE78. Robert J. McEliece. A public-key cryptosystem based on algebraic coding theory. The deep space network progress report 42-44, Jet Propulsion Laboratory, California Institute of Technology, January/February 1978. [https://ipnpr.jpl.nasa.gov/progress\\_report2/42-44/44N.PDF](https://ipnpr.jpl.nasa.gov/progress_report2/42-44/44N.PDF).
- MF06. Payman Mohassel and Matthew Franklin. Efficient polynomial operations in the shared-coefficients setting. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 44–57. Springer, Berlin, Heidelberg, April 2006.
- MGS11. Carlos Aguilar Melchor, Philippe Gaborit, and Julien Schrek. A new zero-knowledge code based identification scheme with reduced communication. In *2011 IEEE Information Theory Workshop, ITW 2011, Paraty, Brazil, October 16-20, 2011*, pages 648–652. IEEE, 2011.
- Nie86. H. Niederreiter. Knapsack-type cryptosystems and algebraic coding theory. *Problems of Control and Information Theory*, 15(2):159–166, 1986.
- NO07. Takashi Nishide and Kazuo Ohta. Multiparty computation for interval, equality, and comparison without bit-decomposition protocol. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007*, volume 4450 of *LNCS*, pages 343–360. Springer, Berlin, Heidelberg, April 2007.
- NY90. Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd ACM STOC*, pages 427–437. ACM Press, May 1990.
- OP01. Tatsuaki Okamoto and David Pointcheval. REACT: Rapid Enhanced-security Asymmetric Cryptosystem Transform. In David Naccache, editor, *CT-RSA 2001*, volume 2020 of *LNCS*, pages 159–175. Springer, Berlin, Heidelberg, April 2001.
- Pat75. N. Patterson. The algebraic decoding of goppa codes. *IEEE Transactions on Information Theory*, 21(2):203–207, 1975.
- Sah99. Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *40th FOCS*, pages 543–553. IEEE Computer Society Press, October 1999.
- SG02. Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. *Journal of Cryptology*, 15(2):75–96, March 2002.
- Sho94. Victor Shoup. Fast construction of irreducible polynomials over finite fields. *Journal of Symbolic Computation*, 17(5):371–391, 1994.

- Sho95. Victor Shoup. A new polynomial factorization algorithm and its implementation. *Journal of Symbolic Computation*, 20(4):363–397, 1995.
- Sho99. Victor Shoup. Efficient computation of minimal polynomials in algebraic extensions of finite fields. In *Proceedings of the 1999 International Symposium on Symbolic and Algebraic Computation*, ISSAC '99, page 53–58. Association for Computing Machinery, 1999.
- Ste94. Jacques Stern. Designing identification schemes with keys of short size. In Yvo Desmedt, editor, *CRYPTO'94*, volume 839 of *LNCS*, pages 164–173. Springer, Berlin, Heidelberg, August 1994.
- THO23. Kota Takahashi, Keitaro Hashimoto, and Wakaha Ogata. Chosen-ciphertext secure code-based threshold public key encryptions with short ciphertext. *Designs, Codes and Cryptography*, Oct 2023.

## A Preliminaries

Here is a description of the constant-round multi-party protocols for elementary algebraic operations that we need in our multi-party decoding algorithm.

- **FieldInv**: this protocol computes a shared field element  $[a^{-1}]$  given a shared non-zero field element  $[a]$  with one secure multiplication.
- **FirstPowers**: this protocol takes a non-negative integer  $k$  and a shared non-zero field element  $[b]$  and compute all the powers  $[b^2], [b^3], \dots, [b^k]$ . Such a protocol is given in [BIB89] as a fan-in multiplication of  $k$  non-zero elements, but can be adapted straightforwardly. It requires  $\mathcal{O}(k)$  secure multiplications.
- **EqTesting**: this protocol aims at proving that a shared element  $[a]$  is zero. While many solutions exist for this protocol, see [DFK<sup>+</sup>06,NO07], the underlying field must be of large characteristic, which is not our case. For our needs, it will be sufficient to reveal publicly the value of  $[ra] = [r] \times [a]$  for a non-zero random element  $r$ . This would need  $\mathcal{O}(1)$  secure multiplications.
- **PolyMul**: Mohassel and Franklin proposed in [MF06] a protocol for multiplying two shared polynomials of degree  $n$  in  $\mathbb{F}_q[X]$  that only require  $\mathcal{O}(n)$  secure multiplications in  $\mathbb{F}_q$ . Note that this protocol enables the multiplication of two elements of a field extension of  $\mathbb{F}_q$  of degree  $n$  with the same complexity, if the irreducible polynomial defining the extension is public.
- **PolyDiv**: this protocol, also provided in [MF06], allows to compute the quotient and the remainder of two shared polynomials  $[f]$  and  $[g]$  with  $\deg f \geq \deg g$  using only  $\mathcal{O}(\deg f)$  secure multiplications. Note that together with the previous protocol, this enables the multiplication of elements of an extension field that is described with a shared irreducible polynomial.
- **FastEval**: this protocol aims at computing the evaluations of a shared polynomial of  $\mathbb{F}_q[X]$  of degree less than  $d$  on a set of  $d$  shared elements of  $\mathbb{F}_q$ . The best constant-round solution to date was proposed recently in [GLOV24]. It needs only  $\mathcal{O}(\tau \cdot d^{1+1/\tau})$  shared products and  $\mathcal{O}(\tau)$  rounds for any constant parameter  $\tau$ .
- **RatRecon**: given two shared polynomials  $[u]$  and  $[g]$ , this protocol computes the two shared polynomial  $[v]$  and  $[r]$  such that  $uv = r \pmod{g}$ , with

degree constraints on  $r$  and  $v$ . For our need, as pointed out in [THO23], the exact degree of  $v$  and  $r$  is known in advance if the error is of correct Hamming weight. Following classical results on gcd and subresultant, see [GG03, section 5.7], this computation can be done through the inversion of a well-chosen subresultant matrix of  $u$  and  $g$ . Using a result from [CD01], this can be done with  $\mathcal{O}(d^3)$  secure products where  $d = \deg g$ .

## B Proof of theorem 2

*Proof.* Let  $\mathcal{S}$  be a simulator for  $(\mathcal{P}, \mathcal{V})$  and let  $\mathcal{T}$  be a one-time simulation extractor for  $(\mathcal{P}, \mathcal{V})$ . Let  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$  be an IND-CCA2<sup>+</sup> adversary running in time at most  $t$  against  $(\mathcal{K}', \mathcal{E}', \mathcal{D}')$ , and making at most  $q_O$  queries. For  $b \in \{0, 1\}$ , we denote by  $\mathbf{Exp}^{real}(\mathcal{A}, b)$  the game  $\mathbf{Exp}^{\text{IND-CCA2}^+}(\mathcal{A}, b)$ . First, let us define the game  $\mathbf{Exp}^{\mathcal{S}}(\mathcal{A}, b)$  as follows:

Game  $\mathbf{Exp}^{\mathcal{S}}(\mathcal{A}, b)$ :

- $(pk, sk) \leftarrow \mathcal{K}(1^\lambda)$
- $(st, \mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}^{O(\cdot, pk, sk), G}(pk)$
- $c^* \leftarrow \mathcal{E}(\mathbf{m}_b, pk)$
- $\pi^* \leftarrow \mathcal{S}^{G_{write}}(pk, c^*)$
- $b^* \leftarrow \mathcal{A}^{O(\cdot, pk, sk), G}(st, (c^*, \pi^*), pk)$
- output  $b^*$

The difference between  $\mathbf{Exp}^{\mathcal{S}}(\mathcal{A}, b)$  and  $\mathbf{Exp}^{real}(\mathcal{A}, b)$  is that in  $\mathbf{Exp}^{\mathcal{S}}(\mathcal{A}, b)$ ,  $\mathcal{S}$  generates the proof corresponding to the challenge ciphertext instead of  $\mathcal{P}$ . Let us show that the two games cannot be distinguished with probability greater than  $\epsilon_{ZK}$ .

**Lemma 2.**

$$|\Pr[\mathbf{Exp}^{\mathcal{S}}(\mathcal{A}, b) = 1] - \Pr[\mathbf{Exp}^{real}(\mathcal{A}, b) = 1]| \leq \epsilon_{ZK}.$$

*Proof.* From  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ , let us build an algorithm  $\mathcal{A}'$  distinguishing between a real proof (corresponding to the game  $\mathbf{ZK}^{real}(\mathcal{A}')$ ) and a simulated proof (corresponding to the game  $\mathbf{ZK}^{\mathcal{S}}(\mathcal{A}')$ ).  $\mathcal{A}'$  works as follows:

1. First,  $\mathcal{A}'$  runs  $\mathcal{K}(1^\lambda)$  to obtain a pair of keys  $(pk, sk)$  and then runs  $\mathcal{A}_1$  on  $pk$ . Whenever  $\mathcal{A}_1$  queries the decryption oracle with randomness,  $\mathcal{A}'$  can answer using  $\mathcal{V}^G$  to verify the proof and can decrypt and recover the randomness using  $sk$ , because  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  has recoverable randomness. When  $\mathcal{A}_1$  queries the random oracle  $G$ ,  $\mathcal{A}'$  simply asks the same question to  $G$  and forwards the answer to  $\mathcal{A}_1$ .
2. When  $\mathcal{A}_1$  outputs two messages  $(\mathbf{m}_0, \mathbf{m}_1)$ ,  $\mathcal{A}'$  computes  $c^* \leftarrow \mathcal{E}(\mathbf{m}_b, r_b, pk)$  (using a random  $r_b$  from  $\mathcal{R}$ ), and queries a proof for  $(pk, c^*)$  by providing the witness  $(\mathbf{m}_b, r_b)$ . When  $\mathcal{A}'$  receives the proof  $\pi^*$ , it gives to  $\mathcal{A}_2$  the ciphertext  $(c^*, \pi^*)$  and continues its interaction with  $\mathcal{A}_2$ . When  $\mathcal{A}_2$  asks a decryption query with randomness, again  $\mathcal{A}'$  can answer using  $\mathcal{V}^G$  to verify the proof and using  $sk$  to decrypt and recover the randomness. When  $\mathcal{A}_2$  asks the

random oracle  $G$ ,  $\mathcal{A}'$  simply asks the same question to  $G$  and forwards the answer to  $\mathcal{A}_2$ .

3. when  $\mathcal{A}_2$  is done and outputs a bit  $b^*$ ,  $\mathcal{A}'$  outputs the same value.

First, observe that  $\mathcal{A}'$  runs in time at most  $\tau_{\mathcal{K}} + t + q_O(\tau_V + \tau_{rand} + \tau_{\mathcal{D}}) + \tau_{\mathcal{S}} \leq t_{ZK}$ . We note that when  $\mathcal{A}'$  is run in the game  $\mathbf{ZK}^{real}(\mathcal{A}')$ , the execution of  $\mathcal{A}$  is identical to the one in the game  $\mathbf{Exp}^{real}(\mathcal{A}, b)$ , since the proof returned to  $\mathcal{A}'$  by the game is a real proof (i.e. computed using  $\mathcal{P}$ ). Moreover, when  $\mathcal{A}'$  is run in the game  $\mathbf{ZK}^{\delta}(D)$ , the execution of  $\mathcal{A}$  is identical to the one in the game  $\mathbf{Exp}^{\delta}(\mathcal{A}, b)$ , since the proof returned by the game to  $\mathcal{A}'$  is a simulated proof (i.e. computed using  $\mathcal{S}$ ). Therefore  $|\Pr[\mathbf{Exp}^{\delta}(\mathcal{A}, b) = 1] - \Pr[\mathbf{Exp}^{real}(\mathcal{A}, b) = 1]|$  and  $|\Pr[\mathbf{ZK}^{real}(\mathcal{A}') = 1] - \Pr[\mathbf{ZK}^{\delta}(\mathcal{A}') = 1]|$  are identical, but since  $(\mathcal{P}, \mathcal{V})$  is  $(t_{ZK}, \epsilon_{ZK})$ -zero-knowledge, the probability of  $\mathcal{A}'$  distinguishing between the games  $\mathbf{ZK}^{real}(\mathcal{A}')$  and  $\mathbf{ZK}^{\delta}(\mathcal{A}')$  is less than  $\epsilon_{ZK}$ . Therefore, the probability of  $\mathcal{A}$  distinguishing between  $\mathbf{Exp}^{\delta}(\mathcal{A}, 0)$  and  $\mathbf{Exp}^{real}(\mathcal{A}, 0)$  is also less than  $\epsilon_{ZK}$ . This concludes the proof of lemma 2.  $\square$

Now, let us prove that it is possible to build an IND-CPA attacker  $\mathcal{B}$  against the cryptosystem  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  using  $\mathcal{A}$ .  $\mathcal{B}$  works as follows:

1.  $\mathcal{B}$  first receives a public key  $pk$  coming from  $\mathcal{K}$ .
2.  $\mathcal{B}$  starts the execution of  $\mathcal{A}_1$  with  $pk$  as input.
  - When  $\mathcal{A}_1$  asks a query to the random oracle  $G$ ,  $\mathcal{B}$  forwards the query to the oracle, and sends back the answer to  $\mathcal{A}_1$ .  $\mathcal{B}$  adds both the query and the answer to a list  $\mathcal{Q}(\mathcal{A}^G)$ .
  - When  $\mathcal{A}$  queries the decryption oracle with randomness on a ciphertext  $(c, \pi)$ ,  $\mathcal{B}$  verifies the proof  $\pi$  by running  $\mathcal{V}^G((pk, c), \pi)$ . If the proof is incorrect,  $\mathcal{B}$  sends  $\perp$  back to  $\mathcal{A}_1$ . Else,  $\mathcal{B}$  runs the extractor  $\mathcal{T}((pk, c), \mathcal{Q}(\mathcal{A}^G))$  to obtain  $(m, r)$ , and sends it to  $\mathcal{A}$  as an answer.
3. When  $\mathcal{A}_1$  outputs two challenge plaintexts  $(m_0, m_1)$ ,  $\mathcal{B}$  forwards them to its own challenger and obtains a challenge ciphertext  $c^*$ . Then,  $\mathcal{B}$  runs  $\mathcal{S}^G(pk, c^*)$  to obtain a simulated proof  $\pi^*$  for  $(pk, c^*)$ . Note that  $\mathcal{S}$  might want to modify the answers of oracle  $G$ .  $\mathcal{B}$  is not able to modify the answers of the oracle. However,  $\mathcal{B}$  can simply remember the values that  $\mathcal{S}$  wished to modify. Then, at any point of their execution, if  $\mathcal{S}$  or  $\mathcal{A}$  wish to query the oracle on a answer modified by  $\mathcal{S}$ ,  $\mathcal{B}$  just answers the modified value by  $\mathcal{S}$  that it remembered instead of querying  $G$  for the real value. This effectively makes  $\mathcal{S}$  and  $\mathcal{A}$  behave as if the answers of the oracle were indeed modified.
4.  $\mathcal{B}$  sends  $(c^*, \pi^*)$  to  $\mathcal{A}_2$  and resumes its execution. When  $\mathcal{A}_2$  queries either the random oracle or the decryption oracle,  $\mathcal{B}$  proceeds identically as in step 2, except when  $\mathcal{A}$  queries the decryption on  $(c^*, \pi^*)$ , in that case  $\mathcal{B}$  just returns  $\perp$ . As explained above, if  $\mathcal{A}_2$  queries  $G$  on a value which has been modified by  $\mathcal{S}$ ,  $\mathcal{B}$  answers with the modified value instead of querying  $G$ .
5. When  $\mathcal{A}_2$  stops and outputs a bit  $b^*$ ,  $\mathcal{B}$  outputs the same bit.

Note that the running time of  $\mathcal{B}$  is at most  $t + q_O(\tau_V + \tau_{\mathcal{T}}) + \tau_{\mathcal{S}} \leq t_{cpa}$ . For  $b \in \{0, 1\}$ , we note that as long as  $\mathcal{B}$  is able to answer correctly the decryption queries



made by  $\mathcal{A}$ , then during an execution of the game  $\mathbf{Exp}_{(\mathcal{K}, \mathcal{E}, \mathcal{D})}^{\text{IND-CPA}}(\mathcal{B}, b)$ , the simulation of  $\mathcal{A}$  done by  $\mathcal{B}$  is identical to the execution of  $\mathcal{A}$  in the game  $\mathbf{Exp}^\delta(\mathcal{A}, b)$ . We denote by  $\mathbf{bad}_b$  the event in which at some time during  $\mathbf{Exp}^\delta(\mathcal{A}, b)$ ,  $\mathcal{T}$  would not be able to extract a witness from a valid proof sent by  $\mathcal{A}$  to the decryption oracle. By the previous remark, we have that:

$$|\Pr[\mathbf{Exp}^\delta(\mathcal{A}, b) = 1 | \neg \mathbf{bad}_b] - \Pr[\mathbf{Exp}_{(\mathcal{K}, \mathcal{E}, \mathcal{D})}^{\text{IND-CPA}}(\mathcal{B}, b) = 1 | \neg \mathbf{bad}_b]| = 0 \quad (6)$$

Therefore, we have that:

$$\begin{aligned} & |\Pr[\mathbf{Exp}^\delta(\mathcal{A}, b) = 1] - \Pr[\mathbf{Exp}_{(\mathcal{K}, \mathcal{E}, \mathcal{D})}^{\text{IND-CPA}}(\mathcal{B}, b) = 1]| \\ & \leq |\Pr[\mathbf{Exp}^\delta(\mathcal{A}, b) = 1 | \neg \mathbf{bad}_b] - \Pr[\mathbf{Exp}_{(\mathcal{K}, \mathcal{E}, \mathcal{D})}^{\text{IND-CPA}}(\mathcal{B}, b) = 1 | \neg \mathbf{bad}_b]| \cdot \Pr[\neg \mathbf{bad}_b] \\ & \quad + \Pr[\mathbf{bad}_b] \\ & \leq \Pr[\mathbf{bad}_b] \end{aligned}$$

Let us bound the probability of  $\mathbf{bad}_b$  with the following lemma:

**Lemma 3.** *For any adversary  $\mathcal{A}$ ,*

$$\Pr[\mathbf{bad}_b] \leq \epsilon_s$$

*Proof.* This lemma is proved using the one-time simulation straight-line extractability of  $(\mathcal{P}, \mathcal{V})$  (see definition 7). Let us build an attacker  $\mathcal{C}$  for the game  $\mathbf{Exp}^{\text{simextract}}(\mathcal{C})$  using  $\mathcal{A}$ .  $\mathcal{C}$  works as follows:

1.  $\mathcal{C}$  generates  $(pk, sk)$  using  $\mathcal{K}(1^\lambda)$ . Then,  $\mathcal{C}$  begins an execution of  $\mathcal{A}_1(pk)$ .
  - When  $\mathcal{A}_1$  queries the random oracle  $G$ ,  $\mathcal{C}$  forwards the query to its oracle, and sends back the answer to  $\mathcal{A}_1$ .  $\mathcal{C}$  adds both the query and the answer to a list  $\mathcal{Q}(\mathcal{A}^G)$ .
  - When  $\mathcal{A}_1$  queries the decryption oracle on  $(c, \pi)$ ,  $\mathcal{C}$  verifies the proof using  $\mathcal{V}^G$ . If the proof is incorrect, it sends back  $\perp$ . Else,  $\mathcal{C}$  runs the extractor  $\mathcal{T}((pk, c), \mathcal{Q}(\mathcal{A}^G))$  to obtain  $(\mathbf{m}, r)$ . Then, it checks whether  $\mathcal{E}(\mathbf{m}, r, pk) = c$  (i.e. whether  $(\mathbf{m}, r)$  is a witness for  $(pk, c)$ ). If it is not,  $\mathcal{C}$  outputs  $((pk, c), \pi)$  as the final result of the game and stops. Else,  $\mathcal{C}$  sends back  $m$  as an answer to  $\mathcal{A}_1$  and resumes its execution.
2. When  $\mathcal{A}_1$  outputs two challenge plaintexts  $(\mathbf{m}_0, \mathbf{m}_1)$ ,  $\mathcal{C}$  samples a random  $r^*$  and computes  $c^*$  as  $\mathcal{E}(\mathbf{m}_b, r^*, pk)$ . It then sends  $(pk, c^*)$  to its challenger, and receives back a simulated proof  $\pi^*$ .
3.  $\mathcal{C}$  resumes the execution of  $\mathcal{A}_2$  with  $(x^*, \pi^*)$  as input. When  $\mathcal{A}_2$  queries either the random oracle or the decryption oracle,  $\mathcal{C}$  proceeds identically as in step 1 (except when  $\mathcal{A}_2$  queries the decryption on  $(c^*, \pi^*)$ , in that case  $\mathcal{C}$  just returns  $\perp$ ).
4.  $\mathcal{C}$  returns  $\perp$  if  $\mathcal{A}_2$  did not query a ciphertext for which the proof is correct and for which the extractor cannot extract a witness.

Observe that  $\mathcal{C}$  runs in time at most  $\tau_{\mathcal{K}} + t + q_O(\tau_V + \tau_{\mathcal{T}} + \tau_{\mathcal{E}}) \leq t_s$ . The execution of  $\mathcal{A}$  done by  $\mathcal{C}$  in the game  $\mathbf{Exp}^{simextract}(\mathcal{C})$  is identical to an execution of  $\mathcal{A}$  in  $\mathbf{Exp}^{\delta}(\mathcal{A}, b)$ . Therefore, the probability that  $\mathcal{C}$  wins the game  $\mathbf{Exp}^{simextract}$  is equal to  $\Pr[\mathbf{bad}_b]$ . But since  $(\mathcal{P}, \mathcal{V})$  is  $(t_s, \epsilon_s)$ -one-time simulation straight-line extractable, this probability is less than  $\epsilon_s$ . This concludes the proof of lemma 3.  $\square$

By lemma 2 and 3 as well as the remark above, we have that for  $b \in \{0, 1\}$ :

$$\begin{aligned} |\Pr[\mathbf{Exp}^{real}(\mathcal{A}, b) = 1] - \Pr[\mathbf{Exp}^{\delta}(\mathcal{A}, b) = 1]| &\leq \epsilon_{ZK}, \text{ and} \\ |\Pr[\mathbf{Exp}^{\delta}(\mathcal{A}, b) = 1] - \Pr[\mathbf{Exp}_{(\mathcal{K}, \mathcal{E}, \mathcal{D})}^{\text{IND-CPA}}(\mathcal{B}, b) = 1]| &\leq \epsilon_s. \end{aligned}$$

Moreover, since  $(\mathcal{K}, \mathcal{E}, \mathcal{D})$  is an  $(t_{cpa}, \epsilon_{cpa})$ -IND-CPA cryptosystem, by definition:

$$|\Pr[\mathbf{Exp}_{(\mathcal{K}, \mathcal{E}, \mathcal{D})}^{\text{IND-CPA}}(\mathcal{B}, 0) = 1] - \Pr[\mathbf{Exp}_{(\mathcal{K}, \mathcal{E}, \mathcal{D})}^{\text{IND-CPA}}(\mathcal{B}, 1) = 1]| \leq \epsilon_{cpa}$$

Therefore:

$$|\Pr[\mathbf{Exp}^{real}(\mathcal{A}, 0) = 1] - \Pr[\mathbf{Exp}^{real}(\mathcal{A}, 1) = 1]| \leq \epsilon_{ZK} + \epsilon_s + \epsilon_{cpa} + \epsilon_s + \epsilon_{ZK}$$

Which concludes the proof of theorem 2.  $\square$

## C Proof of lemma 1

*Proof.* If  $\mathbf{wt}(r) = t$ , let  $E = \{i \in \llbracket 1, n \rrbracket \mid r_i \neq 0\}$  and  $Q = \prod_{i \in E} (X - \gamma_i)$ . Remark that  $Q$  is of the appropriate degree and monic. Notice that for  $1 \leq i \leq n$ , either  $r_i = 1$  and then  $Q(\gamma_i) = 0$ , or  $r_i = 0$  and then  $S_r(\gamma_i) = r_i = 0$ . Overall, we have that for  $1 \leq i \leq n$ ,  $S_r(\gamma_i)Q(\gamma_i) = 0$ . By definition of  $F$ , we therefore have that  $F$  divides  $S_r Q$ , thus there exists  $P \in \mathbb{F}_{poly}[X]$  such that  $S_r Q = FP$ , and we have  $\deg P \leq \deg S_r + \deg Q - \deg F \leq t - 1$ . Moreover, since  $S_r(\gamma_i) = 0$  for exactly  $(n - t)$  of the  $\gamma_i$ 's, we have that  $\deg(\gcd(S_r, F)) = n - t$ . Using a well-known result on subresultants and the remainder sequence of Euclidean algorithm, see [GG03, corollary 6.48], this implies that there exists  $U, V \in \mathbb{F}_{poly}[X]$  such that  $(S_r U + FV) \text{ quo } X^{n-t} = 1$  with  $\deg(U) < \deg(F) - (n - t) = t$ ,  $\deg(U) < \deg(S_r) - (n - t) = t - 1$ .

Suppose that there exists  $Q \in \mathbb{F}_{poly}[X]$  monic of degree  $t$ , and  $P, U, V \in \mathbb{F}_{poly}[X]$  of degree  $\leq t - 1$ ,  $< t$  and  $< t - 1$  respectively, such that we have  $S_r Q = FP$  and  $(S_r U + FV) \text{ quo } X^{n-t} = 1$ . Then, for  $1 \leq i \leq n$ ,  $S_r(\gamma_i)Q(\gamma_i) = r_i Q(\gamma_i) = F(\gamma_i)P_1(\gamma_i) = 0$ . Since  $Q$  is of degree  $t$ , then  $Q$  has at most  $t$  roots among  $\{\gamma_i\}_{1 \leq i \leq n}$ . Using that  $r_i Q(\gamma_i) = 0$  for  $1 \leq i \leq n$ , this implies  $r_i = 0$  on at least  $n - t$  positions, i.e.  $\mathbf{wt}(r) \leq t$ .

In addition,  $(S_r U + FV) \text{ quo } X^{n-t} = 1$  implies that  $\deg(S_r U + FV) \leq n - t$ . Moreover,  $\gcd(S_r, F)$  divides  $S_r U + FV$ , thus  $\deg(\gcd(S_r, F)) \leq \deg(S_r U +$

$FV) \leq n - t$ . Since  $F = \prod_{i=1}^n (X - \gamma_i)$ , this means that  $S_r$  evaluates to 0 on at most  $n - t$  elements in  $\{\gamma_i\}_{1 \leq i \leq n}$ . Since  $\forall 1 \leq i \leq n, S_r(\gamma_i) = r_i$ , we have  $\text{wt}(r) \geq t$ .

Overall, we have  $\text{wt}(r) = t$ , which is the desired result.  $\square$

## D Proof of theorem 4

*Proof.* Let  $\mathcal{A}$  be an adversary running in time at most  $t$  making  $q_1$  queries to  $\mathcal{H}_1$  and  $q_2$  queries to  $\mathcal{H}_2$  for the games  $\mathbf{ZK}^\delta$  and  $\mathbf{ZK}^{real}$ . First, let us describe an algorithm  $\mathcal{S}$  that outputs a proof for  $H = (I_{n-k}|T) \in \mathbb{F}_2^{n-k \times n}$  and  $(c_1, c_2)$ :

1. Sample the challenges:
  - $h_1 = (\epsilon_j^{(e)}, \epsilon'_j{}^{(e)}, \rho_j^{(e)})_{e \in \llbracket 1, \tau \rrbracket, j \in \llbracket 1, s \rrbracket} \leftarrow^{\$} (\mathbb{F}_{points}^3)^{s \cdot \tau}$
  - $h_2 = (i^{*(e)})_{e \in \llbracket 1, \tau \rrbracket} \leftarrow \llbracket 1, N \rrbracket^\tau$
2. Sample root seeds and salt:  $\text{seed}^{(e)} \leftarrow^{\$} \{0, 1\}^\lambda$  for  $1 \leq e \leq \tau$ , and  $\text{salt} \leftarrow^{\$} \{0, 1\}^{2\lambda}$ .
3. Compute the players' seeds  $(\text{seed}_i^{(e)})_{i \in \llbracket 1, N \rrbracket}$  using  $\text{TreePRG}(\text{salt}, \text{seed}^{(e)})$ .
4. Simulate the MPCitH protocol: for  $1 \leq e \leq \tau$ :
  - (a) For each player  $i \neq i^{*(e)}$ :
    - $[a_j^{(e)}]_i, [b_j^{(e)}]_i, [\mathbf{a}'_j{}^{(e)}]_i, [\mathbf{b}'_j{}^{(e)}]_i \leftarrow \text{PRG}(\text{salt}, \text{seed}_i^{(e)})$  for each  $j \in \llbracket 1, s \rrbracket$ .
    - if  $i \neq N$ :
      - derive all these shares from  $\text{PRG}(\text{salt}, \text{seed}_i^{(e)})$ :  
 $([c_j^{(e)}]_i, [c'_j{}^{(e)}]_i)_{j \in \llbracket 1, s \rrbracket}, [r_A]_i^{(e)}, [P]_i^{(e)}, [Q]_i^{(e)}, [\mathbf{y}]_i^{(e)}, [V]_i^{(e)}$ .
      - $\text{state}_i^{(e)} \leftarrow \text{seed}_i^{(e)}$ .
    - if  $i = N$ :
      - Sample the following shares uniformly at random:  
 $[r_A]_N^{(e)}, [P]_N^{(e)}, [Q]_N^{(e)}, [\mathbf{y}]_N^{(e)}, [V]_N^{(e)}$ , and  $([c_j^{(e)}]_N, [c'_j{}^{(e)}]_N)_{j \in \llbracket 1, s \rrbracket}$ .
      - $\text{aux}^{(e)} \leftarrow ([r_A]_N^{(e)}, [P]_N^{(e)}, [Q]_N^{(e)}, [\mathbf{y}]_N^{(e)}, [V]_N^{(e)}, ([c_j^{(e)}]_N, [c'_j{}^{(e)}]_N)_{j \in \llbracket 1, s \rrbracket})$ .
      - $\text{state}_N^{(e)} \leftarrow \text{seed}_N^{(e)} \parallel \text{aux}$ .
    - Simulate the computations of player  $i$  (corresponding to round 3), including part of the product verification protocols in order to compute  $([\alpha_j^{(e)}]_i, [\beta_j^{(e)}]_i, [\alpha'_j{}^{(e)}]_i, [\beta'_j{}^{(e)}]_i)_{j \in \llbracket 1, s \rrbracket}$ .
  - (b) For player  $i^*$  and for  $1 \leq j \leq s$ :  
 $[\alpha_j^{(e)}]_{i^*}, [\beta_j^{(e)}]_{i^*}, [\alpha'_j{}^{(e)}]_{i^*}, [\beta'_j{}^{(e)}]_{i^*} \leftarrow^{\$} \mathbb{F}_{points}^2 \times (\mathbb{F}_{points}^t)^2$ .
  - (c) For each player  $i \neq i^*$ , simulate the rest of the product verification protocols in order to compute  $([v_j^{(e)}]_i, [v'_j{}^{(e)}]_i)$  for  $1 \leq j \leq s$ .
  - (d) For player  $i^*$  and for  $1 \leq j \leq s$ :  
 $[v_j^{(e)}]_{i^*} \leftarrow - \sum_{i \neq i^*} [v_j^{(e)}]_i$  and  $[v'_j{}^{(e)}]_{i^*} \leftarrow 1 - \sum_{i \neq i^*} [v'_j{}^{(e)}]_i$ .
5. Compute the commitments:
  - For  $1 \leq e \leq \tau$ ,  $\text{com}_{i^{*(e)}}^{(e)} \leftarrow^{\$} \{0, 1\}^{p_0(\lambda)}$
  - For  $1 \leq e \leq \tau$ ,  $i \neq i^{*(e)}$ ,  $\text{com}_i^{(e)} \leftarrow \mathcal{H}_0(\text{salt}, e, i, \text{state}_i^{(e)})$ .
6. Set the correct responses for the oracles:

- $\mathcal{H}_1(H, c_1, c_2, \text{salt}, (\text{com}_i^{(e)})_{e \in \llbracket 1, \tau \rrbracket, i \in \llbracket 1, N \rrbracket}) \leftarrow h_1$ .
  - $\mathcal{H}_2(H, c_1, c_2, \text{salt}, h_1, ([\alpha_j^{(e)}, \beta_j^{(e)}, v_j^{(e)}, \alpha'_j{}^{(e)}, \beta'_j{}^{(e)}, v'_j{}^{(e)}]_{e \in \llbracket 1, \tau \rrbracket, j \in \llbracket 1, s \rrbracket})) \leftarrow h_2$ .
7. Output the proof:

$$\text{salt} | h_1 | h_2 | \left( (\text{state}_i^{(e)})_{i \neq i^*(e)} | \text{com}_{i^*(e)}^{(e)} | ([\alpha_j^{(e)}, \beta_j^{(e)}, \alpha'_j{}^{(e)}, \beta'_j{}^{(e)}]_{i^*(e)})_{j \in \llbracket 1, s \rrbracket} \right)_{e \in \llbracket 1, \tau \rrbracket}.$$

Let us prove that  $\mathcal{S}$  simulates a response that is  $(\tau \cdot \epsilon_{PRG} + q_1/2^{2\lambda} + q_2/2^{2\lambda})$ -indistinguishable (in the game  $\mathbf{ZK}^\delta$ ) from a response produced by an honest prover that receives the same challenges (in the game  $\mathbf{ZK}^{real}$ ). To this end, we describe a sequence of hybrid simulators. When dealing with the shares of the proofs, we omit the notations related to the execution  $1 \leq e \leq \tau$ .

**Simulator 0.** This simulator is the honest prover. It takes as input  $H = (I_{n-k}|T) \in \mathbb{F}_2^{n-k \times n}$ ,  $(c_1, c_2)$ , and  $r_A$ . It runs the honest execution of the protocol and outputs the responses of the honest prover.

**Simulator 1.** This simulator can modify the responses of the oracles. It acts identically as simulator 0, except it begins by sampling  $h_1, h_2$  and uniformly at random, and then request the oracles  $\mathcal{H}_1$  and  $\mathcal{H}_2$  to have the correct values for the proof to be accepted (similarly to step 1 and 6 of  $\mathcal{S}$ ). This simulator aborts if at least one of preimage for which the values are modified were already requested by the attacker. This happens with probability at most  $q_1/2^{2\lambda} + q_2/2^{2\lambda}$ . When this simulator does not abort, the value  $\text{com}_{i^*}$  for is uniformly generated instead of being requested to  $\mathcal{H}_0$  (similarly to step 5 of  $\mathcal{S}$ ). Therefore, the output distribution is the same as the one of simulator 0, as the requests of the oracles are independent from any over value and are randomly distributed. Overall, the difference of the output distributions of simulator 0 and 1 is at most  $(q_1 + q_2)/2^{2\lambda}$ .

**Simulator 2.** This simulator acts identically as the previous one, except that the shares of player  $i^*$  corresponding to the input of the MPCitH protocol (corresponding to step 2 of round 1.1 in figure 5) are generated in a uniformly random way instead of using the  $PRG$ . If  $i^* = N$ ,  $([c_j]_N, [c'_j]_N)_{j \in \llbracket 1, s \rrbracket}$ ,  $[r_A]_N, [P]_N, [Q]_N, [\mathbf{y}]_N$ , and  $[V]_N$  are generated as in the protocol. Only the shares of player  $N$  corresponding to  $[a_j]$ ,  $[b_j]_N$ ,  $[\mathbf{a}'_j]$ , and  $[\mathbf{b}'_j]_N$  (for  $1 \leq j \leq s$ ) are generated using true randomness. By the property of the pseudorandom generator, the distribution of the output of this simulator is at most  $\tau \cdot \epsilon_{PRG}$ -indistinguishable from the one of simulator 1 (since this is done for the  $\tau$  executions of the MPCitH protocol).

**Simulator 3.** This simulator does not take  $r_A$  as an input anymore. It acts like simulator 2, except that  $([c_j]_N, [c'_j]_N)_{j \in \llbracket 1, s \rrbracket}$ ,  $[r_A]_N, [P]_N, [Q]_N, [\mathbf{y}]_N$ , and  $[V]_N$  are chosen uniformly at random, and the  $[v_j]_{i^*}$ 's and  $[v'_j]_{i^*}$ 's are computed as  $[v_j]_{i^*} \leftarrow -\sum_{i \neq i^*} [v_j]_i$  and  $[v'_j]_{i^*} \leftarrow 1 - \sum_{i \neq i^*} [v'_j]_i$ . Let us prove that the output distributions of simulator 2 and 3 are identical.

If  $i^* = N$ , the only impacted distributions that appear in the proof are the ones of  $[\alpha_j]_{i^*}$ ,  $[\beta_j]_{i^*}$ ,  $[v_j]_{i^*}$ ,  $[\alpha'_j]_{i^*}$ ,  $[\beta'_j]_{i^*}$ , and  $[v'_j]_{i^*}$  (for  $1 \leq j \leq s$ ). We note that the distribution of  $[\alpha_j]_{i^*}$  and  $[\beta_j]_{i^*}$  are uniform and independent from the other values present in the responses (since  $[a_j]_{i^*}$  and  $[b_j]_{i^*}$  are also uniformly random and independent from any other value). This was already the

case in simulator 2. The same remark is true for  $[\alpha'_j]_{i^*}$  and  $[\beta'_j]_{i^*}$ . Moreover, the relations  $[v_j]_{i^*} = -\sum_{i \neq i^*} [v_j]_i$  and  $[v'_j]_{i^*} = 1 - \sum_{i \neq i^*} [v'_j]_i$  are still true, so the distribution of the  $[v_j]_{i^*}$ 's and  $[v'_j]_{i^*}$ 's are unchanged.

If  $i^* \neq N$ , the only impacted distributions are the one of the elements in **aux** (and of the values that depend on these elements). In simulator 2, we had the following relation:  $[r_A]_N = r_A - \sum_{i \neq N} [r_A]_i = r_A - [r_A]_{i^*} - \sum_{i \neq N, i^*} [r_A]_i$ . Since no value in the response of simulator 1 depended on  $[r_A]_{i^*}$  except for  $[r_A]_N$  and that  $[r_A]_{i^*}$  was uniformly random, then  $[r_A]_N$  was uniformly random and independent of any other value in the response in simulator 2. This is also the case in simulator 3. By similar arguments for all the other shares in **aux**, it is easy to show that **aux** has the same distribution in both simulators (i.e. uniformly random).

**Simulator 4.** This simulator is  $\mathcal{S}$ . The only difference with simulator 3 is that the values  $[\alpha_j]_{i^*}$ ,  $[\beta_j]_{i^*}$ ,  $[\alpha'_j]_{i^*}$ , and  $[\beta'_j]_{i^*}$  are directly drawn uniformly at random instead of being computed from the inputs of player  $i^*$ . However, the values  $[a_j]_{i^*}$  and  $[b_j]_{i^*}$  are already uniformly random and nothing except  $[\alpha_j]_{i^*}$  and  $[\beta_j]_{i^*}$  respectively depend on these values in simulator 2. Therefore, the shares  $[\alpha_j]_{i^*}$  and  $[\beta_j]_{i^*}$  are already uniformly random and independent of the other values in the responses in simulator 2. By a similar argument, this is also the case for  $[\alpha'_j]_{i^*}$  and  $[\beta'_j]_{i^*}$ . Therefore, the output distributions of  $\mathcal{S}$  and simulator 3 are identical.

In conclusion, for a triplet  $(H, c_1, c_2)$  that is correctly formed, the probability of  $\mathcal{A}$  to distinguish between a real proof and the proof from  $\mathcal{S}$  is at most  $\tau \cdot \epsilon_{PRG} + (q_1 + q_2)/2^{2\lambda}$ . Therefore, since in both  $\mathbf{ZK}^{\mathcal{S}}$  and  $\mathbf{ZK}^{real}$ , the prover (be it the honest prover or  $\mathcal{S}$ ) are called only on correctly formed instance  $(H, c_1, c_2)$ , the advantage of  $\mathcal{A}$  for distinguishing the two games is at most  $\tau \cdot \epsilon_{PRG} + (q_1 + q_2)/2^{2\lambda}$ .  $\square$

## E Proof of theorem 5

*Proof.* Before proving the theorem, we need to estimate the probability that an incorrect polynomial equation is flagged as correct for a given proof execution.

**Lemma 4.** *For a proof  $\pi$  and an execution  $e \in \llbracket 1, \tau \rrbracket$ , we say that  $[r_A]^{(e)}$ ,  $[P]^{(e)}$ ,  $[Q]^{(e)}$ ,  $[\mathbf{y}]^{(e)}$ ,  $[V]^{(e)}$  form a good witness of the protocol whenever the values  $r_A^{(e)}$ ,  $P^{(e)}$ ,  $Q^{(e)}$ ,  $\mathbf{y}^{(e)}$  and  $V^{(e)}$  are such that  $S_r^{(e)}Q^{(e)} = FP^{(e)}$  and  $(S_r^{(e)}U^{(e)} + FV^{(e)}) \text{ quo } X^{n-t} = 1$  (where  $S_r^{(e)}$  is built by interpolation from  $r_A^{(e)}$  and  $U^{(e)}$  is built from  $\mathbf{y}^{(e)}$ ). If the values form a bad witness (i.e. not a good witness), the probability of  $(v_j^{(e)})_{j \in \llbracket 1, s \rrbracket}$  to be all zeroes and  $\mathbf{v}^{(e)} = \mathbf{1}$  is bounded by  $p = (\frac{n+t}{\Delta})^s$ , where  $\Delta = \#\mathbb{F}_{points}$  and  $s$  is the number of evaluation point  $\rho_j$ , if  $t-1 \leq n$ .*

*Proof.* We omit the notations related to the execution  $e$ . If the values form a bad witness, then either  $S_r Q \neq FP$  or  $(S_r U + FV) \text{ quo } X^{n-t} \neq 1$ .

If  $S_r Q \neq FP$ , let us evaluate the probability that for  $1 \leq j \leq s$ , we have  $v_j = 0$ . This can happen only if the product verification fails (this event has probability

$1/\Delta$ ) or if  $\rho_j \in \mathbb{F}_{points}$  (which is sampled uniformly) is a root of  $S_r Q - FP$  (i.e.  $S_r Q(\rho_j) = FP(\rho_j)$ ). Moreover,  $S_r Q_1 - FP_1$  is a nonzero polynomial of degree at most  $n - 1 + t$ . Therefore, the probability of  $\rho_j$  to be a root of  $S_r Q - FP$  is at most  $(n + t - 1)/\Delta$ . Overall, the probability for  $v_j$  to be 0 is at most  $(n + t - 1)/\Delta + 1/\Delta = (n + t)/\Delta$ . Over the  $s$  independent values  $v_j$ , the probability that  $(v_j^{(e)})_{j \in \llbracket 1, s \rrbracket}$  is all zeroes is upper bounded by  $((n + t)/\Delta)^s = p$ .

If  $(S_r U + FV) \text{ quo } X^{n-t} \neq 1$ , then for  $1 \leq j \leq s$ , we have that  $v'_j = 1$  if  $\rho_j$  is root of  $(S_r U + FV) \text{ quo } X^{n-t} - 1$  (which is of degree  $2t - 2$ ), or that the product verification fails (which happens with probability  $1/\Delta$ ). The probability that  $\rho_j$  is a root of this polynomial is at most  $(2t - 2)/\Delta \leq (n + t - 1)/\Delta$ . Therefore, the probability that  $v'_j = 1$  is at most  $(n + t)/\Delta$ . Over the  $s$  independent values  $v'_j$ , the probability that  $\mathbf{v}'$  is  $\mathbf{1}$  is upper bounded by  $((n + t)/\Delta)^s = p$ .

In both cases, the probability of a false positive (i.e.  $v_j = 0$  for all  $j$  and  $\mathbf{v}' = \mathbf{1}$ ) for the verification is upper bounded by  $p$ .  $\square$

We can now prove the main theorem. During this proof, we denote by  $(H, c_1, c_2)$  the public key and the ciphertext for which an adversary wants to see a proof, and by  $\pi$  the simulated proof that is given to the adversary. We denote by  $((\tilde{H}, \tilde{c}_1, \tilde{c}_2), \tilde{\pi})$  the final output given by the adversary.

The extractor  $\mathcal{T}$  works as follows: on input of a couple of a public key and a ciphertext  $((\tilde{H}, \tilde{c}_1, \tilde{c}_2), \tilde{\pi})$  and the list of queries  $\mathcal{Q}$  made by the adversary to all the oracles,  $\mathcal{T}$  searches for a query  $\tilde{h}_2$  to  $\mathcal{H}_2$  of the following form and with the following properties:

$$\mathcal{H}_2(\tilde{H}, \tilde{c}_1, \tilde{c}_2, \tilde{\text{salt}}, \tilde{h}_1, (([\tilde{\alpha}_j, \tilde{\beta}_j, \tilde{v}_j])_{j \in \llbracket 1, s \rrbracket}^{(e)}, [\tilde{\alpha}'^{(e)}, \tilde{\beta}'^{(e)}, \tilde{v}'^{(e)}]_{e \in \llbracket 1, \tau \rrbracket})).$$

1.  $\tilde{h}_1$  was the answer to  $\mathcal{H}_1$  of a query of the form:

$$\tilde{h}_1 = \mathcal{H}_1(\tilde{H}, \tilde{c}_1, \tilde{c}_2, \tilde{\text{salt}}, (\tilde{\text{com}}_i^{(e)})_{e \in \llbracket 1, \tau \rrbracket, i \in \llbracket 1, N \rrbracket}).$$

2. There exists  $e^* \in \llbracket 1, \tau \rrbracket$  such that for all players  $i \in \llbracket 1, N \rrbracket$  of the MPC-in-the-head protocol,  $\tilde{\text{com}}_i^{(e^*)}$  was the answer of a query to  $\mathcal{H}_0$  of the form:

$$\tilde{\text{com}}_i^{(e^*)} = \mathcal{H}_0(\tilde{\text{salt}}, e^*, i, \tilde{\text{state}}_i^{(e^*)}).$$

3. The vector  $\tilde{r}$  derived from  $\{\tilde{\text{state}}_i^{(e^*)}\}_{i \in \llbracket 1, N \rrbracket}$ , i.e.

$$\tilde{r} = (\tilde{r}_A | \tilde{r}_B), \text{ with } \tilde{r}_A = \sum_{i=1}^N [\tilde{r}_A]_i, \text{ and } \tilde{r}_B = \tilde{c}_1 - H \tilde{r}_A$$

is such that  $\mathbf{wt}(\tilde{r}) = t$ .

Note that when using hash tables, the extractor runs in time at most  $\mathcal{O}(q_2 + q_1 + q_0 + \tau N)$ . Note that once  $\mathcal{T}$  found such a query,  $\tilde{r}$  is a witness that  $(\tilde{H}, \tilde{c}_1, \tilde{c}_2)$

is well formed. So  $\mathcal{T}$  can output  $\tilde{r}$  as answer (and even output  $\tilde{c}_2 \oplus G(\tilde{r})$  as well if the plaintext is needed).

Let  $\mathcal{A}$  be an adversary for the game  $\mathbf{Exp}^{simextract}$  making at most  $q_0, q_1$ , and  $q_2$  requests to  $\mathcal{H}_0, \mathcal{H}_1$ , and  $\mathcal{H}_2$  respectively. In order to estimate the probability of  $\mathcal{A}$  winning the game, we describe a sequence of hybrid games and estimate the difference between the probability for  $\mathcal{A}$  to win in each game. For the game  $i$ , we denote by  $\Pr[\text{win}_i]$  the probability of  $\mathcal{A}$  winning the game  $i$ . The aim of the proof is to estimate  $\Pr[\text{win}_0]$ . Without loss of generality, in all of the games, we assume that all the requests to the oracles made by the game when verifying the proof  $\tilde{\pi}$  were also made by  $\mathcal{A}$  at some point during the game.

**Game 0.** This game is exactly  $\mathbf{Exp}^{simextract}$ .  $\mathcal{A}$  begins to ask for a simulated proof  $\pi$  for a public key and a ciphertext  $(H, c_1, c_2)$ . Then,  $\mathcal{A}$  outputs another public key-ciphertext pair  $(\tilde{H}, \tilde{c}_1, \tilde{c}_2)$  as well as a proof  $\tilde{\pi}$ .  $\mathcal{A}$  wins if its queries to  $\mathcal{H}_0, \mathcal{H}_1$ , and  $\mathcal{H}_2$  are not enough for  $\mathcal{T}$  to reconstruct a  $\tilde{r} \in \mathbb{F}_2^n$  such that  $\tilde{H}\tilde{r} = \tilde{c}_1$  and  $\mathbf{wt}(\tilde{r}) = t$ .

**Game 1.** This game is identical to Game 0, except it aborts if a collision on  $\mathcal{H}_0$  has been found, i.e. if  $\mathcal{H}_0$  has been queried on two pre-images with the same image by  $\mathcal{H}_0$ . This happens with probability at most  $q_0/2^\lambda$ . Therefore,  $|\Pr[\text{win}_0] - \Pr[\text{win}_1]| \leq q_0/2^\lambda$ .

**Game 2.** This game is identical to Game 1, except it aborts if  $\mathcal{A}$  queries  $\mathcal{H}_1$  on another preimage than the one in the simulated proof  $\pi$ , and if the answer to this query is equal to  $h_1$ . This happens with probability at most  $q_1/\Delta^{2s\tau}$ . Therefore,  $|\Pr[\text{win}_1] - \Pr[\text{win}_2]| \leq q_1/\Delta^{2s\tau}$ .

**Game 3.** This game is identical to Game 2, except it aborts if  $\mathcal{A}$  queries  $\mathcal{H}_2$  on another preimage than the one in the simulated proof  $\pi$ , and if the answer to this query is equal to  $h_2$ . This happens with probability at most  $q_2/N^\tau$ . Therefore,  $|\Pr[\text{win}_2] - \Pr[\text{win}_3]| \leq q_2/N^\tau$ .

Let us estimate the probability  $\Pr[\text{win}_3]$ . If  $\mathcal{A}$  wins game 3, then it means that the proof  $\tilde{\pi}$  is accepted for  $(\tilde{H}, \tilde{c}_1, \tilde{c}_2)$  by the algorithm in figure 6, and that  $\mathcal{T}$  did not find a query verifying the properties 1 to 3. First, suppose that  $\pi = \tilde{\pi}$ , then by the restriction introduced in game 3, we have that  $(\tilde{H}, \tilde{c}_1, \tilde{c}_2) = (H, c_1, c_2)$  (otherwise  $\mathcal{A}$  would have found a collision on  $h_2$ , since  $(H, c_1, c_2)$  are included in the hash). Since it is required that  $((H, c_1, c_2), \pi) \neq ((\tilde{H}, \tilde{c}_1, \tilde{c}_2), \tilde{\pi})$ , we must have  $\tilde{\pi} \neq \pi$ . This means  $\mathcal{A}$  must submit a different proof than  $\pi$ . Moreover, because of the restriction introduced in game 3,  $\mathcal{A}$  cannot reuse the proof  $\pi$  by only modifying the values corresponding to the  $\alpha_j$ 's, the  $\beta_j$ 's, the  $v_j$ 's,  $\alpha'$ ,  $\beta'$ , and  $v'$  in the query to  $\mathcal{H}_2$  and hoping to have  $h_2 = \tilde{h}_2$  (since in this case the game aborts). In the same way, because of the restriction introduced in game 2,  $\mathcal{A}$  cannot reuse the proof  $\pi$  by only modifying some of the commitments in the query to  $\mathcal{H}_1$  and hoping to have  $h_1 = \tilde{h}_1$ . This means that there exists a query made by  $\mathcal{A}$  to  $\mathcal{H}_2$  such that for every execution  $e \in \llbracket 1, \tau \rrbracket$ , one of the following two events is true:

- The extractor  $\mathcal{T}$  cannot recover  $\{\text{state}_i^{(e)}\}_{i \in \llbracket 1, N \rrbracket}$  from queries to  $\mathcal{H}_0$ . This means that  $\mathcal{A}$  did not query  $\mathcal{H}_0$  for the value  $\text{com}_{i(e)^*}^{(e)}$  with a query of the

following form:  $\mathcal{H}_0(\tilde{\text{salt}}, e, i^{*(e)}, \text{state}_{i^{*(e)}}^{(e)})$ , and instead submitted in the proof  $\tilde{\pi}$  a value of its choosing for  $\text{com}_{i^{*(e)}}^{(e)}$ . This means that  $\mathcal{A}$  guessed the correct value  $i^{*(e)}$ . Since this value comes from a part of a query to  $\mathcal{H}_2$ , and that  $\mathcal{H}_2$  is a random oracle, this can only happen with probability at most  $1/N$  (as  $i^{*(e)}$  is uniformly random).

- The witness  $\tilde{r}$  derived from  $\{\text{state}_i^{(e)}\}_{i \in [1, N]}$  by  $\mathcal{F}$  is such that  $\text{wt}(\tilde{r}) \neq t$ . By lemma 1, this means that one of the polynomial equations in 1 is not satisfied, but that the checks corresponding to the  $[v_j^{(e)}]$ 's and  $[v'^{(e)}]$  are correctly verified. By lemma 4, this can only happen with probability at most  $p$ .

Overall, since there are  $\tau$  independent executions in  $\tilde{\pi}$ , and that  $\mathcal{A}$  made at most  $q_2$  queries to  $\mathcal{H}_2$ , we have that:

$$\Pr[\text{win}_3] \leq q_2 \left( p + \frac{1}{N} \right)^\tau.$$

By summing all the inequalities, we have the correct probability for equation 3, which concludes the proof.  $\square$

## F A brief description of code-based threshold PKEs from [THO23]

The first scheme  $\Pi_t^I$  directly applies Dodis-Katz conversion [DK05]: it starts from an IND-CCA2 KEM-DEM relying on Classic-McEliece and an IND-CCA2 secret-key encryption scheme. The message is encrypted using the secret-key encryption with an ephemeral secret key  $k$  divided into  $N$  shares. These shares are encrypted with the secret-key encryption using  $N$  keys produced by the encapsulation mechanism for the different Classic-McEliece public keys. The set of ciphertexts is signed using a one-time signature. The secret key is just the secret key  $sk$  of the Classic-McEliece scheme (whose size is  $(n + t)m$  bits), the ciphertext expansion comes from the  $N$  Classic-McEliece ciphertext  $c_{\text{CM}}$  encapsulating  $N$  keys and the encryptions of the shares of the initial secret key  $k$ , the signature  $\sigma$  and the verification key  $vk$ . The signature can be instantiated from a code-based signature: the authors proposed the one from [BGKM23] whose size for 128-bit security is 15 kB. The decryption is non-interactive.

The second scheme  $\Pi_t^{\text{II}}$  first generically makes a one-way CPA encryption scheme threshold with the same security, and then applies Cong *et al.*'s techniques [CCMS21] to transform a (rigid deterministic) threshold one-way CPA secure encryption into a CCA-secure hybrid encryption in the spirit of REACT [OP01]. The first step generates a blow up of all parameters since the number of keys and ciphertext components are in  $\mathcal{O}(\binom{N-1}{T})$  (because it needs encryption to any possible subsets of  $T - 1$  elements among  $N$ ) which makes it completely impractical when  $N$  grows.



Cong *et al.*'s conversion is interesting when the considered one-way CPA secure encryption has an efficient distributed decryption. That is why in the last scheme  $\Pi_t^{\text{III}}$ , the authors of [DK05] tried to design a distributed decryption for McEliece to remove the dependency in  $\binom{N-1}{T}$ . Note that for both  $\Pi_t^{\text{III}}$  and  $\Pi_t^{\text{II}}$  Cong *et al.*'s conversion requires the secure distributed evaluation of some hash functions (which counts as the  $\mathcal{O}(\binom{N}{T-1}n)$  and the extra  $\mathcal{O}(n)$  in the communication complexity).

All the parameterized sizes for secret keys and ciphertexts are recapitulated in Table 2. Both  $\Pi_t^{\text{I}}$  and  $\Pi_t^{\text{III}}$  use hashed values that are of length  $\ell_g$  and  $\ell_h$ . Note that  $\Pi_t^{\text{I}}$  and  $\Pi_t^{\text{II}}$  have secret key and ciphertext sizes depending on the number of participants  $N$ .